# GeoGuessers: What We Did, Why, and What It Means

Internal report for the team (not for submission)

Phases 0–5 complete (including the interactive viewer and E6). Read this before the presentation.

Swayam Goyal (2024111012) Siddharth Choudhary (2024111021)
Yakkala Viswanadha Kartikeya (2024111033)

## Contents

**1 The project in one page** **3**

**2 Concepts you need (glossary)** **4**

2.1 The terrain operators and their formulas . . . . . 9

2.2 Error measures used in Phase 4 . . . . . 9

**3 What we did, in order** **9**

3.1 Phases 0–2 (setup, data, baseline) . . . . . 9

3.2 Phase 3 (our engine) . . . . . 10

3.3 Phase 4 (pyramid on demand) . . . . . 11

3.4 Audit, reports, plan change and hand-off . . . . . 11

3.5 Phase 5 (the interactive viewer and E6) . . . . . 11

**4 The data** **12**

**5 How the measurements work** **13**

5.1 One run . . . . . 13

5.2 Cold cache . . . . . 13

5.3 Protocol . . . . . 13

5.4 Why we pin to performance cores . . . . . 13

5.5 Why only two folders are excluded from Defender . . . . . 13

5.6 Splitting a gdaldem run into stages . . . . . 13

5.7 GPU safety . . . . . 14

5.8 Experiments . . . . . 14

**6 Phase 2 results: the standard tools** **14**

6.1 E1: how gdaldem scales . . . . . 14

6.2 E2: does the file layout matter? . . . . . 15

6.3 QGIS: why the pyramid matters . . . . . 16

**7 Phase 3 results: our own engine** **17**

7.1 How the engine works . . . . . 17

7.2 Correctness . . . 17
7.3 E3: CPU engine vs gdaldem . . . 17
7.4 E4: the GPU engine . . . 19

**8 Phase 4 results: derivatives on demand from the pyramid (G2) 20**
8.1 Accuracy . . . 20
8.2 Time . . . 22

**9 Phase 5: the interactive viewer and E6 22**
9.1 What changed and how the work was organised . . . 22
9.2 Architecture . . . 23
9.3 Tile engine core (final plan Phase 1) . . . 23
9.4 Scheduler and server (final plan Phase 2) . . . 25
9.5 Web viewer (final plan Phase 3) . . . 25
9.6 QGIS plugin (final plan Phase 4) . . . 26
9.7 E6 method (final plan Phase 5) . . . 27
9.8 E6 results . . . 27
9.9 Policy set from the data . . . 32

**10 The whole story (what to say in the presentation) 33**

**11 Problems we faced and what we learned 33**

**12 Validity and limitations 36**

**13 What is left 37**
13.1 Final deliverables (final plan Phase 6) . . . 37
13.2 Stretch (only if the above is on track) . . . 37
13.3 Schedule and owners (relative weeks) . . . 37
13.4 Open items . . . 37
13.5 Questions to ask the professor . . . 37

**14 Presentation preparation 38**
14.1 Numbers to remember . . . 38
14.2 Suggested slide order . . . 39
14.3 Likely questions and answers . . . 39

**15 How to reproduce everything, and where things are 42**

# 1 The project in one page

**The question.** Our professor asked us to work like a systems team: take a very large, high-resolution DEM (Himalaya, 90 m down to 1 m), use only a laptop, and find out (a) where today's standard tools (GDAL, QGIS) stop being practical and *why*: what is the real bottleneck (disk, RAM, CPU, GPU transfer, rendering)? And (b) how to process and display only what the user is looking at, using a **pyramid** (the same DEM stored at several resolutions), computing slope / aspect / hillshade **on demand**, and using the CPU and GPU where each really helps. Our literature review turned this into these research gaps:

**G1** No study reports where standard GDAL/QGIS terrain tools stop being practical on one laptop, separated into stages (read, compute, CPU↔GPU transfer, write, display).

**G2** No study computes terrain derivatives on demand from pyramid levels, or measures how wrong a coarse-level slope is against the time it saves.

**G3** (stretch) Accuracy of fast approximate viewshed (XDraw) vs exact R3 in high relief; A* vs Dijkstra.

**What is finished.** Phase 0 (setup), Phase 1 (data), Phase 2 (baseline of the standard tools, E1/E2/QGIS), Phase 3 (our own CPU and GPU engine, E3/E4), Phase 4 (derivatives on demand from the pyramid, E5), all audited number by number against the raw data. Phase 5: the interactive viewer (a tile server, a browser viewer with continuous debounced pan/zoom and neighbour prefetch, a QGIS plugin), built and measured in E6 (1,068 runs). What remains are the final deliverables: final report, slides, demo.

**The answer, in seven lines.**

1. `gdaldem` (GDAL's terrain tool, which QGIS also calls) **never runs out of memory**: it handles 10.8 billion cells (43 GB) with 0.87 GB of RAM. It just gets slower: 3.5 min for one slope pass at 1 m.
2. Its bottleneck is **computation on a single CPU thread** (73% of the time; 1 of 22 threads used).
3. For **display**, the bottleneck is **reading full-resolution data**: without a pyramid QGIS needs 32 s to draw the whole 1 m area; with a pyramid 0.09 s at any size.
4. **Our CPU engine** computes slope **2.1–3.5× faster** than `gdaldem` on the same cores (1 m: 60 s vs 208 s), identical results, memory flat at ∼1.4 GB. Loading the whole grid into RAM (the naive way) breaks above roughly 0.5–0.7 billion cells.
5. **Our GPU kernel** computes 50× faster than the CPU kernel, but the run is only 1.0–1.4× faster (1 m: 44 s): 95% of the GPU's own time is copying, and the run is limited by decompressing the input and writing the output. **The bottleneck moved from computation to data movement.**
6. **On demand from the pyramid**, one display tile costs 5–8 ms at every zoom level, 8–450× less than computing it exactly at full resolution. The price: slope comes out *too low*, by 2.3° RMSE one level up to 6.4° four levels up (worst on steep slopes). Aspect is barely affected.
7. **Our viewer** computes every $512^2$ tile on demand while the user pans and zooms continuously. With neighbour prefetch 98–100% of the tiles entering the view while panning are already in the browser; a 150 ms trailing debounce cuts the server's tiles per zoom gesture from 62 to 19 (3.3×); prefetch does not slow the visible tiles. The remaining latency comes from decoding blocks and the browser's 6 connections, not from computing.

**Where we go next.** The final deliverables (the final plan's Phase 6, Section 13): final report, slides and a recorded demo of the viewer and the QGIS plugin as a fallback for the live demo.

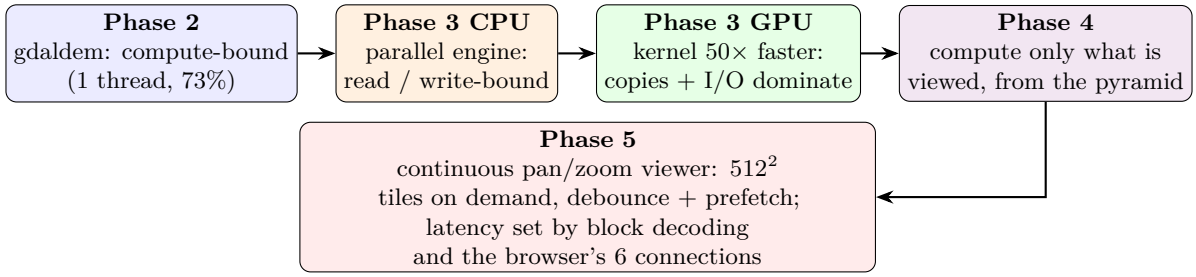



**Figure 1:** The story of the project in one picture. Each step removes the current bottleneck and exposes the next one. The answer to "how do we handle a DEM bigger than RAM on a laptop" is: do not process all of it; read and compute only the window and resolution the user needs, and (Phase 5) do that fast enough to follow a user panning and zooming.

## 2 Concepts you need (glossary)

| Term | Meaning, and why it matters for us |
|---|---|
| *Data and formats* | |
| DEM | Digital Elevation Model: a grid (raster) where each cell stores a ground height. |
| Resolution vs cells | Halving the cell size quadruples the number of cells. Our area at 8 m is 169 M cells; at 2 m 2.7 B; at 1 m 10.8 B. Run time follows the number of cells, which is why we talk in cells, not metres. |
| float32 / float64 | 4 / 8 bytes per number. 10.8 B cells × 4 B = 43 GB, 2.6× our 16 GB RAM. float32 is what the files store; float64 is more precise arithmetic. |
| MB vs MiB | MB = $10^6$ bytes, MiB = $2^{20}$ (5% more). Our harness records MiB; our documents use MB/GB. GPU memory is given in MiB/GiB as the driver reports it. (Mixing these was one of the audit fixes.) |
| CRS, UTM 44N | Coordinate reference system. Slope needs the cell size in metres, so everything is reprojected to UTM zone 44N (EPSG:32644). Degrees would give wrong slopes. |
| Resampling | Moving a grid onto a new grid. *Nearest* copies the closest value (keeps values, blocky), *bilinear*/*cubic* blend neighbours (smooth), *average* averages all cells that fall into a coarser cell (used to build pyramids). |
| GeoTIFF, strips, tiles | The file format. Data stored in rows (strips) or squares (tiles, e.g. 512×512). Tiles let a program read a small window without reading whole rows. |
| Compression, predictor | LZW, DEFLATE, ZSTD shrink the file; the price is CPU time to decompress on every read. The floating-point predictor (=3) makes height data compress much better. |
| COG | Cloud-Optimized GeoTIFF: a tiled GeoTIFF with its pyramid stored inside. All our prepared grids are COGs (512 tiles, ZSTD). |
| Overview / pyramid level | A coarser copy stored inside the file. Level $k$ has cell size $\approx 2^k\times$ base (2 m → 4, 8, 16 … m). GDAL halves the size and rounds down, so when the grid size is not divisible by $2^k$ a level cell is slightly bigger than $2^k$ fine cells (8.002 instead of 8). |

| Term | Meaning, and why it matters for us |
|---|---|
| Vertical datum | What height zero means. HMA 8 m and Chamoli 2 m use the WGS84 ellipsoid; Copernicus the EGM2008 geoid: about 24.5–26 m apart here. A constant offset does not change slope, but matters for viewsheds. |
| NMAD | Normalised median absolute deviation: a robust standard deviation used to compare DEMs. |
| Synthetic grid | Our 2 m and 1 m grids of the whole area are *cubic upsamples* of the 8 m DEM. They have the right size but no real extra detail: used for timing only, never accuracy. |
| *Terrain operators* | |
| 3×3 operators | Slope, aspect, hillshade, TRI, TPI, roughness each look at a cell and its 8 neighbours. One visit per cell: O(N). Faster versions differ in implementation, not in the maths. |
| Horn vs ZT | Two ways to estimate the height gradient (Section 2.1). Horn uses all 8 neighbours with weights; Zevenbergen–Thorne only the 4 direct ones. GDAL's default is Horn. |
| Slope / aspect / hillshade | Steepness in degrees; direction the slope faces (0–360° from north); shaded relief lit by a sun at azimuth 315°, altitude 45° (grey value 1–255, 0 = nodata). |
| TRI, TPI, roughness | Terrain Ruggedness Index (Riley: root of summed squared differences; Wilson: mean absolute difference), Topographic Position Index (cell minus mean of neighbours), max − min of the window. |
| Nodata | Cells without a value (−9999). An operator output is nodata if any of the 9 window cells is nodata; `gdaldem` also leaves the outer ring of the grid empty. |
| *Measurement* | |
| File cache, cold / warm | Windows keeps recently read file data in spare RAM (the standby list). Cold: we empty it first, so the file must come from disk. Warm: immediate repeat, data may come from RAM. |
| Peak working set | The most RAM a process held at any moment: our "peak memory". |
| ns per cell | Run time / number of cells, in nanoseconds: makes grids of different sizes comparable. |
| Median, IQR, spread | We repeat runs and report the median (robust to one odd run). IQR = 75th − 25th percentile; spread = (max − min) / median. |
| Warm-up run | rep 0: a first run that is thrown away (loads programs and caches). |
| P-, E-, LP-cores | Our CPU has 6 fast performance cores (12 threads), 8 slower efficiency and 2 very slow low-power cores. Where Windows puts a process changes its speed by up to 2.7×; we pin tools to the P-cores. |
| Turbo budget | Cores share a power/heat budget: if other cores are busy, the active core clocks lower. |
| Defender scan, `$Deleted` | Windows Defender scans new files; a file deleted while being scanned keeps its space reserved in a hidden NTFS area until the scan ends. |
| Stage split | Dividing a run into read / compute / write (or copy / kernel for the GPU) to see where time goes. |
| Amdahl's law | If a part $p$ of a run is sped up infinitely, the whole run speeds up at most $1/(1-p)$. Read + write = 27% of gdaldem ⇒ parallelising only compute gives at most ∼3.7×. |
| Read-bound / write-bound | The run's speed is set by reading (decompressing) input or writing output, not by computation. |
| *Our engine (CPU)* | |

| Term | Meaning, and why it matters for us |
|---|---|
| Window, halo | The grid is processed in 2048×2048 windows; each is read with a 1-cell border (halo) so the 3×3 operators are exact at window edges. |
| Pipeline | Three stages that overlap: reader threads decode windows, the main thread computes, one writer thread writes. Bounded queues (at most 6 windows in flight) keep memory flat. |
| Numba, JIT, `prange` | Numba compiles a Python loop to machine code just in time; `prange` splits the rows over all cores. The compiled kernel is cached on disk. |
| Fused kernel | Slope, aspect and hillshade computed from one gradient pass: three products for little more than one. |
| Naive version | Read the whole grid into memory and compute with NumPy: simple, but needs ∼24 bytes per cell. |
| *Our engine (GPU)* | |
| CUDA kernel, CuPy | A function run by thousands of GPU threads at once (one per cell). We write the kernels in CUDA C and compile them through CuPy at run time. |
| H2D / D2H, PCIe | Copies host→device and device→host over the PCIe bus (here 4.0 ×8). Moving a cell in and out costs ∼0.9 ns; computing it 0.04 ns. |
| Pinned memory | Page-locked host memory: copies run at full PCIe speed and can overlap with kernels. |
| CUDA streams | Independent queues of GPU work; with 2 streams the copies of one window overlap the kernel of the next. |
| Crossover | The window size above which the GPU (including copies) beats the CPU: $512^2$ cells for slope. |
| VRAM, TDR | GPU memory (8 GiB, we capped use at 4 GiB); TDR is Windows' 2 s GPU watchdog (our kernels take ms). |
| *Phase 4 (pyramid)* | |
| On demand | Compute the derivative from the pyramid level being displayed, at that level's cell size. |
| Truth (E5) | Slope computed at full resolution, then averaged over exactly the fine cells a coarse cell covers. |
| Footprint | The fine cells covered by one coarse cell (those whose centres lie inside it). |
| Bias, RMSE, MAE | Mean of (on demand − truth); root mean squared error; mean absolute error. Negative bias = too low. |
| Fully valid cell | A coarse cell whose footprint and 8 neighbours contain no nodata: the only cells we score. |
| Circular mean, $R$ | Average of directions (via sin/cos); $R$ (0–1) says how much the directions agree. |
| Levels up | How many pyramid levels above the source data: the error depends on this, not on metres. |
| *Phase 5: the tile server* | |
| Tile lattice | A fixed grid of $512 \times 512$ tiles per pyramid level, aligned to that level's own $512^2$ COG blocks. A tile is named (level, $t_x$, $t_y$); with its 1-cell halo it touches up to 9 stored blocks. |
| Level choice, hysteresis | The viewer shows the finest level whose cell is at most one screen pixel (real cell sizes, not $2^k$). Hysteresis (±10%) keeps it from flipping between two levels when the zoom sits near a boundary. |
| Foreground / background (fg / bg) | Foreground tiles are what the user is looking at now; background tiles are speculative prefetch. They have separate queues and separate worker threads. |

| Term | Meaning, and why it matters for us |
|---|---|
| GIL, `nogil` | Python's global interpreter lock lets only one thread run Python code at a time. Numba functions compiled with `nogil=True` (and zlib compression) release it, so several tiles really run in parallel on several threads. |
| Palette PNG, Z_RLE | An 8-bit PNG storing one byte per cell plus a 256-colour palette (index 0 transparent = nodata): smaller and faster to encode than RGBA. Z_RLE is zlib's run-length deflate mode: 2–3× faster than default deflate for 5–15% larger tiles. |
| Block cache, result cache | Two server caches: decoded $512^2$ blocks as float32 (1024 MiB; all-nodata blocks stored as a 0-byte marker) and finished PNG tiles (512 MiB), keyed by a BLAKE2b hash of the full tile specification. |
| SLRU | Segmented LRU cache: new entries go to a probation segment (20%); an entry hit again moves to the protected segment (80%). A one-off scan (a prefetch ring never visited) cannot flush entries that are really reused. |
| Single-flight | Identical concurrent requests share one computation: 100 identical requests compute once. |
| Generation | A counter per viewer session, raised at every zoom commit. Work for an older generation is superseded (dropped) on the server. |
| Cancel token | A flag each job checks between stages (before dequeue, after each block read, before the kernel, before encoding); a cancelled job stops at the next check. |
| Promotion | A foreground request for a tile still queued as prefetch moves that job to the foreground queue. |
| Yielding (prefetch) | A background worker does not start a new tile while any foreground tile is queued or running (`DEMLAB_BG_YIELD`, default on). |
| Memory guard, 503 | Prefetch pauses when free RAM falls below 1.0 GB (`DEMLAB_BG_MIN_FREE_GB`); while paused, prefetch requests get HTTP 503 at once and the client backs off. |
| Efficiency class, last-level cache | Windows labels every logical CPU with an efficiency class; on our CPU the E- and LP-cores get the same class, so we tell them apart by which last-level cache they sit behind. |
| Pinning (threads) | Fixing a thread to chosen CPUs (`SetThreadAffinityMask`): foreground workers on the 12 P-core threads (CPUs 0, 1, 10–19), prefetch workers on the 8 E-cores (CPUs 2–9); LP-cores (20–21) unused. |
| `Server-Timing` | An HTTP response header; every tile reports its queue, read, decode, kernel, colour-map and encode times and its cache tier in it. |
| Event-driven disconnect | The server learns that a browser aborted a request from the connection-closed event, instead of checking every 20 ms, and cancels the job. |
| *Phase 5: the viewer* | |
| Commit | The moment the viewer fixes a new view (level and extent) and requests its tiles. |
| Debounce, trailing debounce | Waiting until input has stopped before acting. Trailing: the zoom commits once, 150 ms after the *last* wheel tick, instead of on every tick. Meanwhile the screen shows the tiles it already has, rescaled. |
| `maxWaitMs` | An optional forced commit after continuous scrolling for that long; off (measured to nearly double the tiles). |
| Fallback chain | What a frame draws for a missing tile: a coarser cached level, else a finer cached level, else the overview (the coarsest level, always loaded). So the view is never blank. |

| Term | Meaning, and why it matters for us |
|---|---|
| Prefetch ring | After a commit, the 8 view-sized rectangles around the view (N, S, E, W and the corners) are loaded at background priority, at most 2 requests at a time, aborted when the user goes elsewhere. |
| Velocity-first | Ring order: the direction of the last pan first, then cardinals before corners, then nearer before farther. |
| Recentre | The ring is rebuilt when the camera moves more than 25% of the view off the committed centre (at most once per 100 ms); ring requests no longer needed are aborted. |
| Back-off | The client stops prefetch for 2 s when the p95 of its last 20 foreground tiles exceeds 250 ms (or on a 503). |
| 6 connections per origin | A browser opens at most 6 HTTP/1.1 connections to one server; further tile requests wait for a free one. |
| `AbortController` | The browser mechanism that cancels a fetch; it closes the connection, which the server sees as a disconnect. |
| Box zoom, `POST /view` | Shift + drag zooms the viewer to a box. `POST /view` computes a box directly as a PNG or GeoTIFF; the QGIS plugin uses it. |
| *Phase 5: measuring the viewer (E6)* | |
| Virtual client | A Python port of the viewer's policy (camera, level choice, browser cache, debounce, generations, aborts, prefetch) that replays a trace in real time against the live server with 6 connections and 60 Hz frames. |
| Trace | A seeded, stored script of user input (pans, wheel bursts, notches, box zooms) replayed identically in every run. |
| Notched wheel | A mouse wheel that clicks in discrete steps 30–300 ms apart, unlike the 17 ms gaps of a 60 Hz burst. |
| Pan hit rate | Share of tiles already cached in the browser when they first enter the view during a pan. |
| Prefetch precision | Prefetched tiles later drawn, divided by all prefetched tiles (0.34 means about 3 fetched per 1 used). |
| TTFF, complete | Time to first frame: from the last wheel tick (or the end of a box animation) to the first frame showing a tile of the committed level. *Complete*: to the first frame with every visible tile of that level. |
| Blank area-time | Share of view area × frames not drawn from the needed level's own tiles (a preview or the overview instead). |
| Server tiles per gesture | Foreground jobs the server finished or cancelled mid-render, divided by zoom gestures. |
| p50 / p95 | The median and 95th percentile of tile latencies: 95% of tiles are faster than the p95. |
| Cold start | Every E6 run starts a new server with empty caches, so latencies are on the slow side. |
| Chrome DevTools protocol (CDP) | The interface through which a program drives Chrome. We drive headless Chrome over it in real time with the `websockets` package (no Playwright). *Virtual time* (a headless mode in which the page's clock does not follow real time) distorted every network wait and was dropped. |
| High-resolution event loop | Python 3.12's default event-loop clock on Windows moves in 15.6 ms steps; the virtual client runs its own loop on the high-resolution clock (timers accurate to about 1 ms). |
| RSS | Resident set size: the RAM a process holds (server memory in E6, in MiB). |
| *Experiments* | |

| Term | Meaning, and why it matters for us |
|---|---|
| E1–E6 | E1 gdaldem baseline; E2 storage layouts; E1q QGIS display; E3 CPU engine; E4 GPU engine; E5 pyramid accuracy and time; E6 interactive viewer (Phase 5: virtual client replaying traces against the tile server, 1,068 runs). |

### 2.1 The terrain operators and their formulas

Label the 3×3 window around cell $e$ as

$$\begin{array}{ccc} a & b & c \\ d & e & f \\ g & h & i \end{array} \qquad \text{cell size } \Delta x = \Delta y.$$

**Horn (1981):**

$$\frac{\partial z}{\partial x} = \frac{(c + 2f + i) - (a + 2d + g)}{8\Delta x}, \qquad \frac{\partial z}{\partial y} = \frac{(a + 2b + c) - (g + 2h + i)}{8\Delta y}.$$

**Zevenbergen–Thorne (1987):**

$$\frac{\partial z}{\partial x} = \frac{f - d}{2\Delta x}, \qquad \frac{\partial z}{\partial y} = \frac{b - h}{2\Delta y}.$$

Then

$$s = \arctan\sqrt{(\partial z/\partial x)^2 + (\partial z/\partial y)^2}, \qquad \text{aspect} = \operatorname{atan2}\Big(-\tfrac{\partial z}{\partial x}, -\tfrac{\partial z}{\partial y}\Big)$$

(**slope** $s$; **aspect** is the direction of steepest descent in degrees from north, nodata on perfectly flat cells); **hillshade** (sun azimuth $\phi$, altitude $\theta$): $\cos\gamma = \sin\theta\cos s + \cos\theta\sin s\cos(\phi - \text{aspect})$, value $= 1 + 254\max(\cos\gamma, 0)$, as `gdaldem` does; **TRI** (Riley) $= \sqrt{\sum(z_n - z_e)^2}$, (Wilson) the mean of $|z_n - z_e|$ over the 8 neighbours $n$; **TPI** $= z_e - \text{mean}(z_n)$; **roughness** $=$ max $-$ min of the window.

Three consequences used later: (1) every operator divides by the cell size, so slope computed on a coarser pyramid level is *different* (smoother), not just faster: that is what E5 measures. (2) All six need the same few neighbour differences, so computing them together (one pass) shares most of the work. (3) `gdaldem` adds the neighbour values in float32; our float64 engine therefore differs from it by tiny rounding on nearly flat cells.

### 2.2 Error measures used in Phase 4

For the $n$ scored cells with on-demand value $c_j$ and truth $t_j$: bias $= \frac{1}{n}\sum(c_j - t_j)$, MAE $= \frac{1}{n}\sum|c_j - t_j|$, RMSE $= \sqrt{\frac{1}{n}\sum(c_j - t_j)^2}$. Aspect differences are taken around the circle ($\min(|\Delta|, 360 - |\Delta|)$).

## 3 What we did, in order

### 3.1 Phases 0–2 (setup, data, baseline)

1. **Setup.** Python environment `geo` (GDAL 3.13.3, rasterio, Numba, CuPy 14.2, FastAPI) via Miniforge at `D:\miniforge3` (the default path failed because our user folder name has a space), QGIS 3.44.14. Checked: CuPy runs on the RTX 4060; QGIS slope matched an OpenTopography reference to $10^{-5}$ degrees.

2. **Choosing data.** The first downloads (an Everest 90 m crop, two single 8 m strips) were too small or patchy. We switched to the NASA High Mountain Asia 8 m *mosaic* tile-571 (Uttarakhand), downloaded Copernicus 30 m and 90 m for the same box through the OpenTopography API, and found a free real 2 m DEM of the Chamoli valley inside the tile (Bhushan & Shean, Zenodo).

3. **One area, many resolutions.** Run time depends on the number of cells, not on which mountain it is. One area means grid size is the only thing that changes, and the real 2 m data gives a "truth" for accuracy.

4. **Checking the data.** Tile-571 is 90.4% valid (gaps in the south-east). The 8 m tile is 24.5 m below Copernicus (datum). Chamoli 2 m averaged to 8 m matches the 8 m DEM to a median of −0.7 m (NMAD 5.1 m). Chamoli was re-gridded with nearest-neighbour (only 0.09/0.16 m off our grid), so its heights are untouched.

5. **Are synthetic grids OK?** No free 1 m DEM exists here. On a 16 M-cell crop, an 8 m grid rebuilt from 16 m takes the same gdaldem time as the real one, but its slopes are off by 2.3–3.0° RMSE, and nearest-neighbour data compresses 26% better (fake fast reads). Decision: synthetic 2 m and 1 m, *cubic*, timing only.

6. **Building the harness** (Section 5): exact per-process measurements, cold-cache flushing, resumable sweeps.

7. **First E1 pass** (74 min, 560 runs, 0 failures), then a critical review: the "read" reference was wrong, one run was a 2× outlier, conditions were not recorded.

8. **Fixing the method.** Core test → pin to performance cores. New pure read reference. Defender's CPU use measured (tiny). Run conditions saved with every sweep.

9. **Second attempt** ran into a full disk: 108 GB of deleted outputs held in `$Deleted` while Defender scanned them. We stopped, freed space, added a safeguard (wait until deleted space is released), re-ran.

10. **Chamoli anomaly.** Some runs 25–35% slower in streaks, with foreign disk reads: background scanning of previous outputs. Then E2 (10.8 GB outputs) was stopped by the safeguard itself (35.6 GB held >15 min).

11. **Final setup, clean re-run.** Only the two scratch folders excluded from Defender, laptop restarted, everything re-run: **832 runs, 0 failures**. Disturbed attempts kept as evidence. GDAL tuning check afterwards (`GDAL_NUM_THREADS`, bigger cache): no speed-up, so the baseline is `gdaldem` at its fastest.

### 3.2 Phase 3 (our engine)

12. **Kernels and correctness first.** NumPy reference kernels for all 10 configurations, then a parallel Numba kernel (slope/aspect/hillshade fused). 60 tests against `gdaldem` on two $1024^2$ crops (one with 55% nodata, one steep). Found that `gdaldem` adds the Horn neighbours in float32; with that reproduced, every cell matched.

13. **First engine version**, then three fixes before benchmarking: it submitted all window reads at once (up to ∼86 GB in memory at 1 m) → bounded prefetch; rasterio's "boundless" read was slow → manual padding (8 m slope 1.55 s → 1.12 s); writing in the compute thread blocked the kernel → one writer thread (as the GDAL multi-threading docs advise). Also: Numba used 22 threads while the process was pinned to 12 → explicit thread count.

14. **E3 sweep** (CPU, 347 runs), keeping the D: drive safe: outputs written to the internal SSD and deleted after each run (max one run's output at a time, worst 97 GB). Window-size sweep (84 runs) added because the plan listed tile size. Diagnostics (kernel alone in memory, reader threads) as one-off scripts.

15. **Consolidation:** the 7 remaining `gdaldem` configurations benchmarked (154 runs, total E3 = 501) and a full-grid correctness check (every cell of the whole 8 m and 2 m grids).
16. **GPU part** (with the explicit rule not to stress the laptop): CUDA kernels for all 10 configurations, pinned buffers, 2 streams, CUDA-event timing, a 4 GiB memory cap, and a temperature guard in the harness (wait below 75 °C, stop at 85 °C). E4 sweep 484 runs (0 failures, max 63 °C) plus a crossover test.

### 3.3 Phase 4 (pyramid on demand)

17. **Discovered** that GDAL's overviews are not exact halvings on our grids (rounding), so a coarse cell's footprint is taken from its geometry; `demlab/pyramid.py` with 11 tests.
18. **E5 accuracy:** slope and aspect from every overview level vs the full-resolution truth, on both real DEMs, fully valid cells only; exact power-of-two pyramid as a cross-check; slope classes; thresholds; independent DEMs.
19. **E5 timing:** one display tile and the whole grid per level, three routes (on demand / full resolution / precomputed), cold and warm, in one long-running process. Precomputed slope pyramids built and costed.

### 3.4 Audit, reports, plan change and hand-off

20. **Full audit of Phases 2–4:** raw data integrity (1817 runs, no duplicates, only 2 expected failures), every report regenerated (identical), every number in every document checked by independent read-throughs. Fixed: a table that mixed cold and warm runs, MiB/GB units, several overstated claims (Section 11). Added: correctness check on the 90 m/30 m grids (the plan's "correctness gate"), median + IQR for every configuration, stage chart.
21. **LaTeX progress reports** for Phases 3 and 4 (Phase 2 report corrected).
22. **Team decision for Phase 5:** zoom by bounding boxes instead of streaming tiles. *Superseded* by the professor's input (item 24): box zoom survives only as Shift + drag and for QGIS.
23. **Hand-off:** everything pushed to GitHub; a complete SHA-256-verified copy (project, all data, environment, notes) on the external drive `K:\spatial project\GeoGuessers_handoff`.

### 3.5 Phase 5 (the interactive viewer and E6)

24. **Professor's input:** the viewer should work like a map, with continuous pan and zoom and prefetch of the neighbourhood, not box by box. Kartikeya wrote the engineering spec (`docs/PHASE5_DEBOUNCED_ZOOM_SPEC.md`: goals, architecture, E6 gates); we wrote `docs/FINAL_PLAN.md`, which renumbers the remaining work (Phase 0 = everything above; Phases 1–5 = the viewer and E6; Phase 6 = final deliverables) and adopts the spec where it is the best choice and differs where noted. Team decisions: continuous debounced pan/zoom is the main interaction; box zoom stays as Shift + drag and for QGIS; owners: backend and server Swayam, web viewer and prefetch Kartikeya, E6 Siddharth, QGIS plugin and final deliverables shared.
25. **Tile engine core** (final plan Phase 1): tile lattice, GIL-free tile kernels, block and result caches, palette PNG; 24 new tests (143 in total). Parallel gate first failed at 4.3×; passed at 5.4× after moving colour quantisation and PNG encoding off the GIL.
26. **Scheduler and server** (Phase 2): two pinned worker pools, single-flight, generations, cancel tokens, FastAPI server; 17 new tests (160). Isolation gate noisy (+6–15%) until prefetch was made to yield; cancellation gate failed with polling (58–80 ms) and passed event-driven (37 ms).

27. **Web viewer** (Phase 3): plain ES modules, debounce, fallback chain, prefetch ring, box zoom; 6 Node unit tests and an in-browser self-test in headless Chrome. Fixes: memory guard default, 503 for paused prefetch, pending prefetch flag, double commit on box zoom, blank first map.
28. **QGIS plugin** (Phase 4): box tool → `POST /view` → styled GeoTIFF layer; headless end-to-end check 13/13; server limit of 64 M cells per box (161 tests).
29. **E6** (Phase 5): virtual client, 15 seeded traces, pilot (back-off threshold raised to 250 ms); first sweep stopped after 17 runs because of timer precision, rerun with a high-resolution loop: 996 runs in 147 min; then 2 notched-wheel traces (72 runs) that set the debounce to 150 ms; headless-Chrome cross-check. Total 1,068 runs.
30. **Write-up:** results document, tables and charts (`docs/PHASE5_RESULTS.md`; `results/e6_tables.md`; `results/plots/e6_*.png`); the Phase 5 LaTeX progress report (`report/phase5_report`); and this report.

# 4 The data

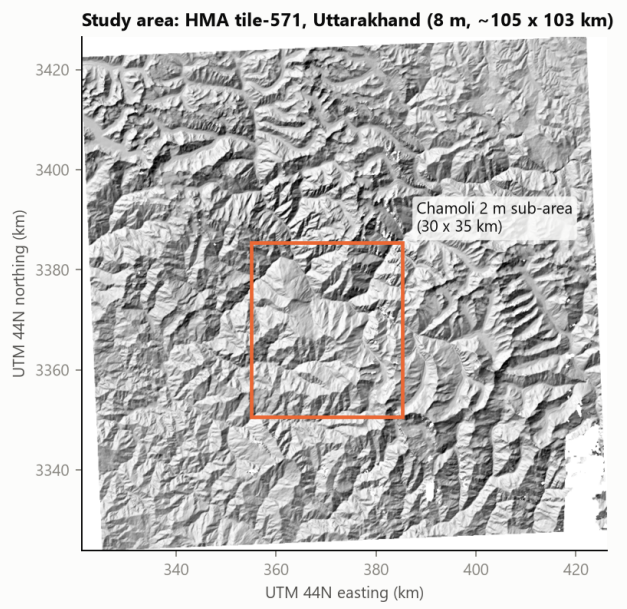



**Figure 2:** Study area (8 m shaded relief) and the Chamoli 2 m sub-area.

| File (`D:\geoguessers_data`) | Res. | Cells | Valid | Notes |
|---|---|---|---|---|
| `prepared/tile571_90m.tif` | 90 m | 1.3 M | 100% | Copernicus GLO-90 |
| `prepared/tile571_30m.tif` | 30 m | 12 M | 100% | Copernicus GLO-30 |
| `prepared/tile571_8m.tif` | 8 m | 169 M | 84% | HMA mosaic; reprojection leaves thin empty wedges at the tilted edges |
| `prepared/chamoli_2m.tif` | 2 m | 264 M | 94% | real 2 m; heights identical to the download; accuracy truth |
| `synthetic/tile571_2m_cubic.tif` | 2 m | 2.7 B | ∼89% | timing only, 5.3 GB on disk |
| `synthetic/tile571_1m_cubic.tif` | 1 m | 10.8 B | ∼89% | timing only, 17.9 GB on disk (incl. pyramid) |

All grids share one origin on a 360 m lattice (divisible by 90, 30, 8, 2 and 1), so they line up exactly. The raw downloads are in `datasets/`; everything is listed with checksums in `datasets/manifest.csv` and rebuilt by `python datasets/prepare.py [--synthetic]`. Other data folders: `e2/` (8 m layout copies; the 2 m ones are on the external drive), `bench_out/` (scratch; holds the Phase 4 slope pyramids), `test_fixtures/` (unit-test crops), `tools/` (RAMMap).

# 5 How the measurements work

## 5.1 One run

`bench/harness.py` starts the tool (e.g. `gdaldem slope in.tif out.tif`, or `python -m demlab.cli ...` for our engine) as its own process, pinned to a set of cores, and waits. When it ends, the harness reads from Windows, for that process only: wall time, peak memory, total CPU time (so CPU time / wall time = average cores used), bytes read and written. It also counts bytes physically read from the disks: if the disk read much more than the process did, someone else was using the disk (our "foreign disk read" interference signal). Our engine additionally prints its own stage timings (read wait, compute, write wait, writer busy; GPU copy/kernel times) as JSON, which go into the `extra` column. One row per run in `results/runs.csv`. An early version sampled counters every 50 ms and missed the end of short runs; reading exact totals from the finished process fixed that.

## 5.2 Cold cache

Emptying the Windows file cache needs admin rights, so a scheduled task (`GeoGuessers-FlushCache` in the `\GeoGuessers` folder of the Task Scheduler) runs Sysinternals `RAMMap -Et` with admin rights; the benchmark triggers it without being admin. Verified: after a flush the whole file is read from disk again. (An early mistake used `-Ec`, which is not a RAMMap option.)

## 5.3 Protocol

Real grids: 5 cold runs (cache emptied before each), then 1 warm-up and 5 warm runs. Synthetic 2 m/1 m: 3 cold + 1 warm (saved ∼5 hours; operator comparisons do not depend on size, so they come from the real grids). Medians are reported; median and IQR of every configuration are in `results/run_stats.md` (median IQR 1.2–1.6% of the median for E1–E4). Plugged in, Balanced power plan, Defender on (only scratch folders excluded), recorded per sweep in `results/env/`.

## 5.4 Why we pin to performance cores

The same 8 m slope job pinned to each of the 22 logical CPUs: performance cores 3.7–4.1 s, efficiency cores ∼6.6 s (1.7×), low-power cores up to 10.5 s (2.7×). Unpinned, Windows may move a background process onto slow cores mid-run (one first-pass run took 108.9 s instead of ∼56 s). Pinning runs every baseline tool at its best, so our speed-ups are *conservative*. Our engine was measured both on the same 12 P-core threads (fair comparison) and on all 22.

## 5.5 Why only two folders are excluded from Defender

Defender's CPU use during a run is tiny (0.02–0.05 core). The problem is *after* the run: it scans each new 10–40 GB output, and a file deleted meanwhile keeps its space reserved until the scan ends. That filled the disk once (108 GB) and slowed later runs. Excluding just our two scratch folders loses no protection. The harness also waits until deleted space is released before the next run and stops cleanly if that takes more than 15 minutes.

## 5.6 Splitting a gdaldem run into stages

Two reference runs per grid: **read** (`bench/read_blocks.py`: read and decompress every block, nothing else) and **copy** (`gdal_translate` to an uncompressed GeoTIFF: same read plus same-size write). Then write + tool overhead ≈ copy − read, and compute ≈ gdaldem − copy. (The first read reference, `gdalinfo -checksum`, spent half its time computing the checksum.)

### 5.7 GPU safety

Our rule was not to put the graphics card or the laptop at risk. So: CUDA kernels only through CuPy (no clock, power-limit, fan or driver changes); GPU memory capped at 4 GiB; before every GPU run the harness reads the GPU temperature and waits until it is below 75 °C, and stops the sweep if a run ends at 85 °C or more; every kernel is milliseconds, far below the 2 s Windows watchdog. Result: max 63 °C, median 58 °C, max 2.0 GiB.

### 5.8 Experiments

| Experiment | What was run |
|---|---|
| E1 (560 runs) | 10 `gdaldem` configurations on 6 grids, cold and warm, plus read/copy references. At 1 m only slope and hillshade (Horn). |
| E2 (224 runs) | 7 storage layouts of the 8 m and 2 m grids: time to write, pure read, `gdaldem` slope. |
| E1q (48 runs) | QGIS (headless) opens the layer and draws one 1920×1080 image, whole area or zoomed, with or without the pyramid. |
| E3 (501 runs) | CPU engine, 24 variants on 8 m / 2 m real / 2 m / 1 m synthetic: 22 vs 12 threads; all 10 configurations; fused products; with/without writing; compressed output; 1–22 threads; windows 512–8192; single-threaded NumPy kernel; naive whole-grid NumPy. |
| E4 (484 runs) | GPU engine, same grids: optimisation steps (CuPy port → own kernel → pinned → 2 streams), float32 vs float64, all operators, fused products, windows, 4 vs 8 readers. Plus a crossover test (one in-memory window, $128^2$–$8192^2$). |
| E5 | Accuracy of slope/aspect from every pyramid level (Chamoli 2 m: 6 levels; 8 m: 5 levels) on fully valid cells, exact-pyramid cross-check, slope classes, thresholds, independent DEMs. Timing: one $512^2$ tile at 8 places and the whole grid per level, three routes, cold/warm (670 timings summarised). |
| E6 (1,068 runs) | The viewer: a virtual client replays 17 seeded traces (pans, zoom bursts, notched wheels, zoom-pan, box zooms) against a fresh tile server per run; 24 configurations, one factor at a time; 1 warm-up + 5 measured runs each. Plus a headless-Chrome cross-check on 3 traces (Section 9.7). |

## 6 Phase 2 results: the standard tools

### 6.1 E1: how gdaldem scales

**Left chart.** Both axes logarithmic; a straight 45° line means "time proportional to cells". Slope and hillshade are straight from 12 M cells up: **linear scaling**, about 19–25 ns per cell. At 90 m the line bends because a run lasts 0.07 s, mostly program start-up. Slope: 4.0 s (8 m), 6.6 s (2 m real), 54 s (2 m synthetic), 208 s (1 m).

**Right chart.** gdaldem's memory *stops growing* at ∼0.87 GB while the grid grows past the 16 GB line: gdaldem reads a few rows, computes, writes, moves on (streaming), keeping only GDAL's block cache (5% of RAM) plus the program. **Meaning:** gdaldem cannot "break" on memory here. "Where does it break?" has the answer "it doesn't; it becomes slow". The real breaking points are memory-hungry approaches (Phase 3's naive version) and display.

**Cold vs warm** differ by 2% or less: reading is a small part of the run on this NVMe SSD (cold reads measured at 2.7–3.3 GB/s), and at 2 m/1 m even warm runs read 3.8/14.2 GB from disk (at 1 m the file exceeds the cache; at 2 m gdaldem's own ∼11 GB output pushes the input out). Our

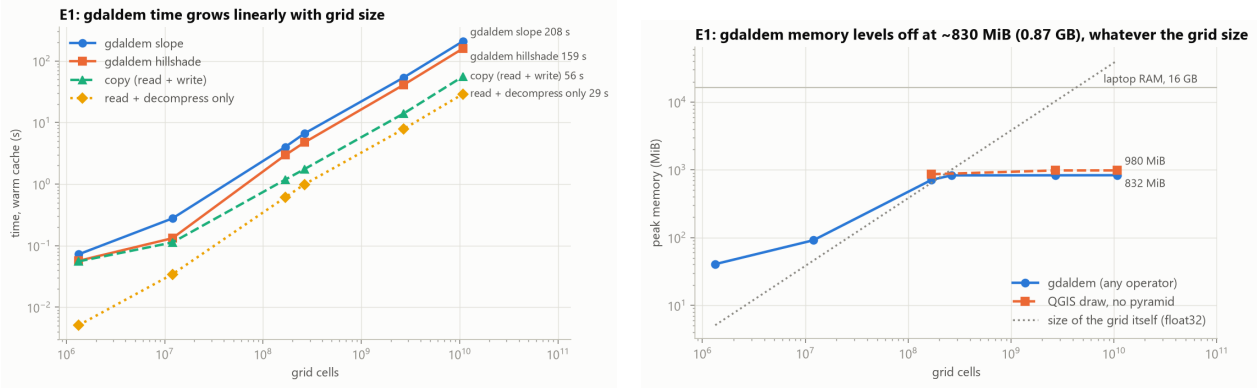



**Figure 3:** Left: run time vs cells. Right: peak memory vs cells.

pilot (lit review) saw 3×; it was timed through QGIS on another laptop, so its first runs most likely included loading the programs.

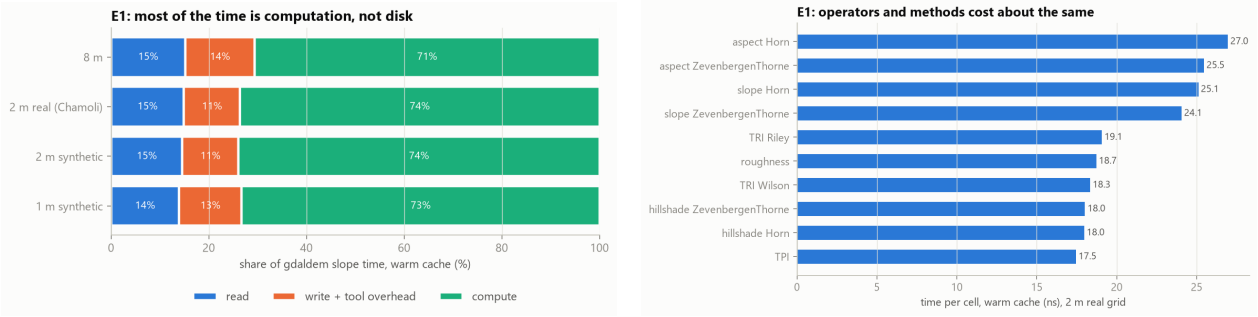



**Figure 4:** Left: stage split of a gdaldem slope run. Right: cost per cell of the ten configurations (2 m real).

**Stage split.** At every real size ∼15% is reading + decompressing, 11–14% writing, **71–74% computing**. This contradicts the common claim that data movement dominates: for gdaldem on a fast SSD it is the arithmetic on one thread. Amdahl: parallelising only the computation gives at most ∼3.7×; going further needs parallel decompression and less writing.

**Operators.** All ten cost 17.5–27 ns per cell; Horn vs ZT differ by at most 6%. The formula is not where speed comes from; implementation and data movement are. **Pinning:** on 2 m synthetic, unpinned cold runs ranged 55.8–108.9 s (74% spread); pinned 53.7–55.2 s (2.8%).

## 6.2 E2: does the file layout matter?

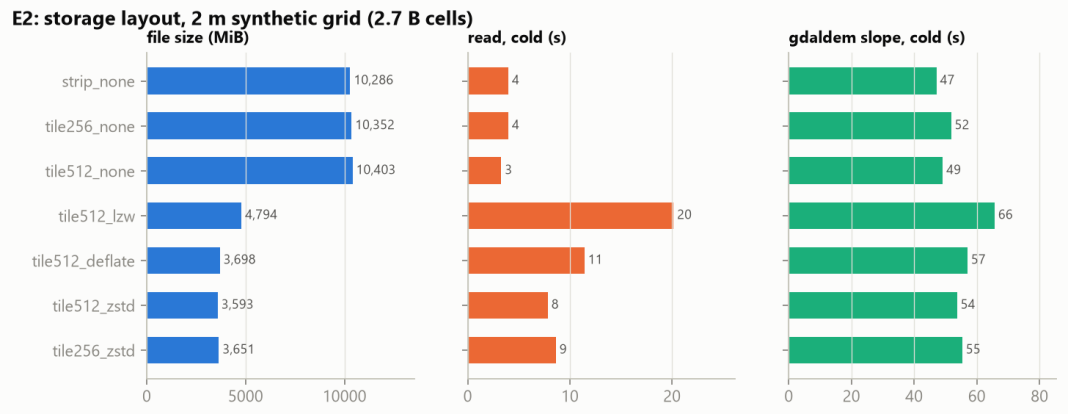



**Figure 5:** File size, read time and gdaldem slope time per storage layout (2 m synthetic, 2.7 B cells).

- **Size:** uncompressed 10.8–10.9 GB; LZW 5.0 GB; DEFLATE 3.9 GB; ZSTD 3.8 GB (2.9× smaller).
- **Read:** uncompressed 3.3–4.0 s; ZSTD 7.8 s; DEFLATE 11.4 s; LZW 20.1 s (decompression costs CPU).
- **gdaldem slope:** uncompressed strips 47 s (fastest), ZSTD 54 s, DEFLATE 57 s, LZW 66 s (+39%), because gdaldem decompresses inside its single thread.
- **Write:** compressed files take 6–9× longer to create (ZSTD 124 s vs 14 s): a one-time cost.
- **Tiles vs strips:** strips are fine for a row-by-row pass; tiles matter for reading small windows (the viewer); 512 slightly better than 256.

**Meaning:** layout changes gdaldem time by up to ∼40%. Tiled ZSTD (our COGs) is the right compressed format; avoid LZW.

### 6.3 QGIS: why the pyramid matters

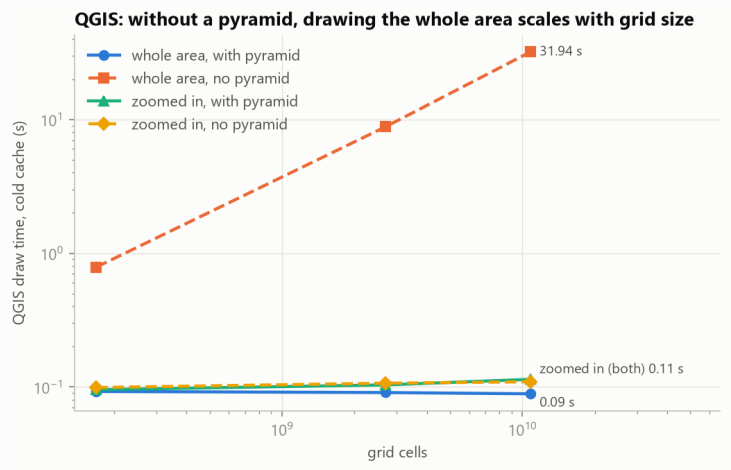



**Figure 6:** QGIS: time to draw one 1920×1080 view.

- **Whole area, with pyramid:** 0.09 s at 8 m, 2 m and 1 m (peak 0.21 GB): QGIS reads only the level whose cells match the screen pixels.
- **Whole area, no pyramid:** 0.79 s, 8.8 s, **31.9 s** (peak ∼1.03 GB): it decompresses the whole full-resolution level (about 12.7 GB at 1 m), close to the 29 s a pure read takes. 360× slower at 1 m.
- **Zoomed in:** ∼0.1 s with or without pyramid (only the window is read). **Opening:** 0.02–0.03 s.

**Meaning:** the display bottleneck is how much full-resolution data a view must read. A pyramid makes zoomed-out views cost the same at any grid size, which is the meeting's proposed design.

**Key idea.** Phase 2 answer to G1: on this laptop the standard tools do not fail at 1 m, they stop being practical. Processing is limited by single-threaded computation; display by reading full-resolution data without a pyramid.

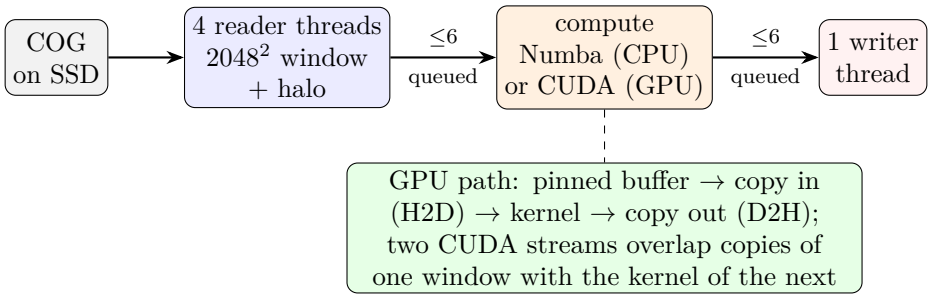



**Figure 7:** The engine: a bounded three-stage pipeline. Memory stays flat whatever the grid size.

# 7 Phase 3 results: our own engine

## 7.1 How the engine works

`demlab/engine.py` (pipeline), `demlab/kernels/numba_k.py` (CPU), `demlab/kernels/cuda_k.py` and `demlab/gpu.py` (GPU), `demlab/cli.py` (command line used by the benchmarks). All ten `gdaldem` configurations; slope, aspect and hillshade fused.

## 7.2 Correctness

119 unit tests and a full-grid check: CPU and GPU engines vs `gdaldem` on **every cell** of the whole 90 m, 30 m, 8 m and 2 m grids, all 10 configurations (1.3 M to 247 M valid cells per grid). Nodata masks identical and all values within tolerance. The only exception: our float64 CPU engine differs in Horn aspect on nearly flat cells (up to 0.08% of cells) and Horn slope on one cell (0.022°), because `gdaldem` adds the Horn neighbours in float32. With that rounding reproduced, every cell of all four grids matches; the float32 GPU kernel matches with no exception.

## 7.3 E3: CPU engine vs gdaldem

**Table 3:** Slope (Horn), warm cache. "Same cores" = the 12 performance-core threads gdaldem used.

| Grid | Cells | gdaldem | Engine, same cores | Speed-up | Engine, 22 thr. | Peak memory |
|---|---|---|---|---|---|---|
| 8 m | 169 M | 4.02 s | 1.94 s | 2.1× | 2.28 s | 1.35 GB |
| 2 m real | 264 M | 6.63 s | 2.53 s | 2.6× | 2.72 s | 1.37 GB |
| 2 m synthetic | 2.7 B | 53.7 s | 16.3 s | 3.3× | 16.4 s | 1.42 GB |
| 1 m synthetic | 10.8 B | 208.4 s | **60.3 s** | **3.5**× | 68.2 s | 1.43 GB |

- **2.1–3.5× faster on the same cores, growing with size** (1 m: 5.6 vs 19 ns per cell). Small grids gain less because ∼1 s of every run is fixed cost (Python start-up 0.5–0.6 s, loading the compiled kernel 0.45 s).
- **12 P-core threads beat all 22** (1 m: 60 s vs 68 s): slow cores hold the evenly split work up.
- **Memory flat** at ∼1.4 GB (gdaldem 0.87 GB).
- **Breaking point of the naive approach:** ∼24 bytes per cell (4.1 GB at 8 m, 6.4 GB at 2 m real). At 2 m synthetic it held 9.6 GB when a further 10.7 GB allocation was refused (whole run would need ∼65 GB); at 1 m the first allocation (43 GB) was refused. Whole-grid processing stops at roughly 0.5–0.7 B cells on 16 GB.
- **Speed comes from the compiled parallel kernel**: the same pipeline with a single-threaded NumPy kernel is 2.6–3× *slower* than gdaldem.

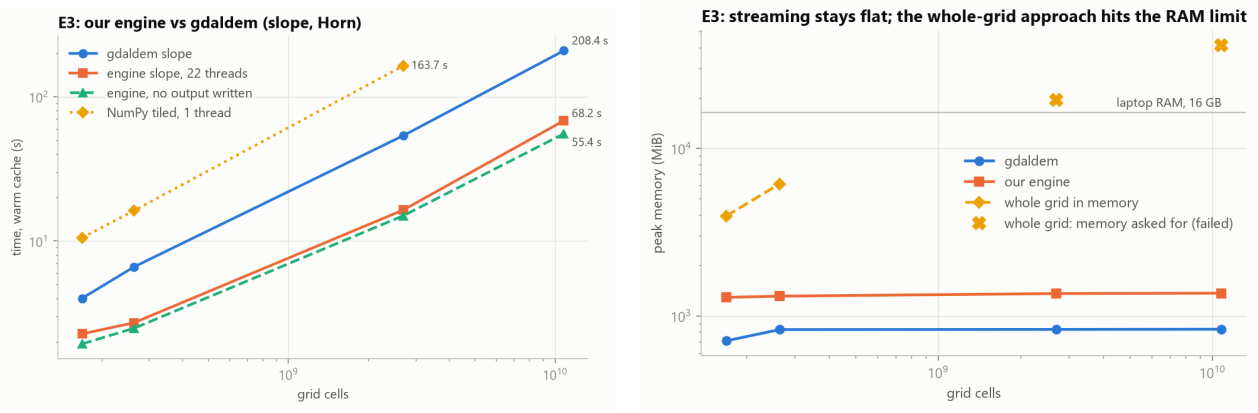



**Figure 8:** Left: engine vs gdaldem. Right: memory: the engine is flat; the naive whole-grid version fails.

- **Every operator faster**: all 10 configurations 1.3–2.0× at 8 m, 1.7–2.6× at 2 m real.
- **Three products in one pass**: 3.3 s vs 11.3 s for three gdaldem runs at 8 m (3.4×), 4.4 s vs 18.5 s at 2 m real (4.2×); 135 s at 1 m.

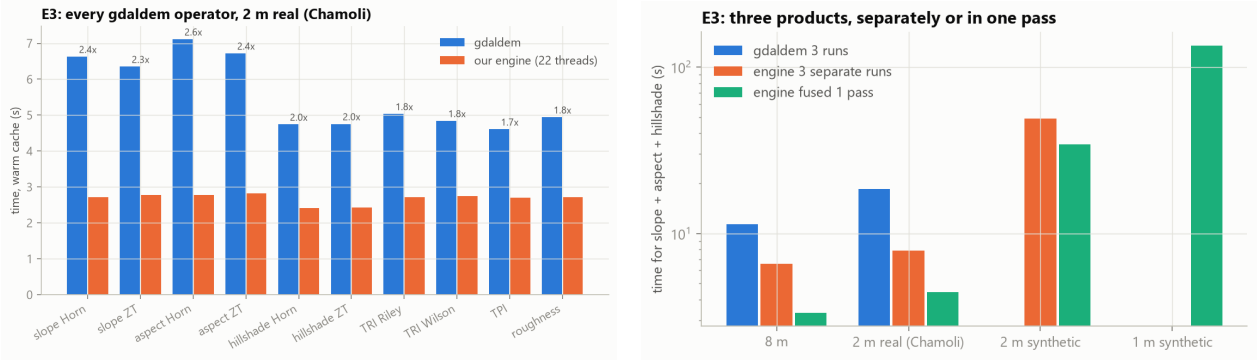



**Figure 9:** Left: all ten configurations (2 m real). Right: three products separately or fused.

**Where the time goes** (slope, 22 threads, warm, 1 m): total 68.2 s; kernel 33 s; the single writer thread is busy 65 s writing 43 GB (∼0.66 GB/s): **write-bound**. Without writing (55.4 s) it is **read-bound**: 23 s waiting for windows to be decompressed. Compressed output (ZSTD) is 40% smaller but slower (21.2 vs 16.4 s at 2 m synthetic). Cold vs warm: almost no difference.

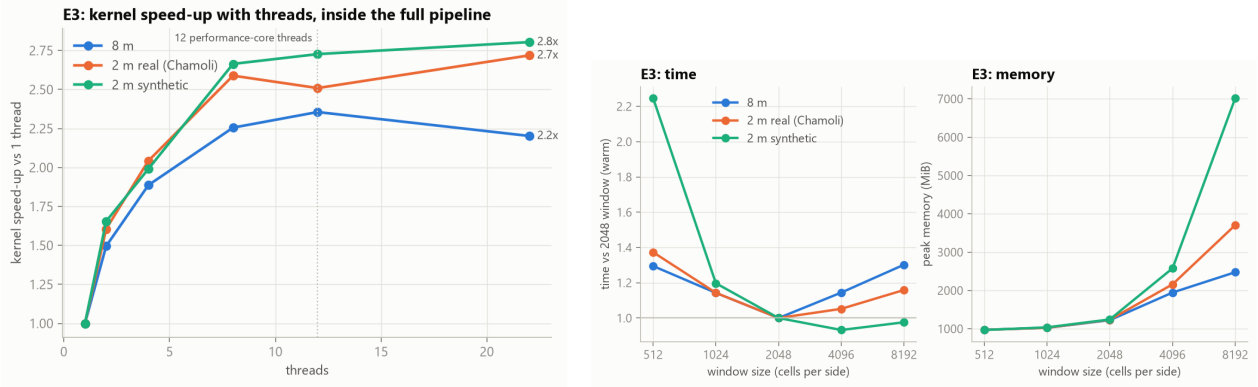



**Figure 10:** Left: kernel speed-up with threads inside the pipeline. Right: window size.

**Threads:** inside the pipeline the kernel speeds up only 2.2–2.8× from 1 to 22 threads; alone in memory it scales to ∼16 threads (9.5 → 1.7 ns per cell). It shares cores with decompression and writing. **Window size:** 2048 is best. Small windows are slow because of the halo: the one-cell

border forces the neighbouring compressed 512-tiles to be decoded again, $(w/512+2)^2$ instead of $(w/512)^2$ tiles: 9× extra work at 512, 2.25× at 2048. An 8192 window holds 7.4 GB in flight for no gain.

### 7.4 E4: the GPU engine

**Table 4:** Slope (Horn), warm: gdaldem, CPU engine (12 threads), GPU engine.

| Grid | gdaldem | CPU engine | GPU engine | GPU vs CPU | GPU vs gdaldem | GPU, no output |
|---|---|---|---|---|---|---|
| 8 m | 4.02 s | 1.94 s | 1.90 s | 1.0× | 2.1× | 1.80 s |
| 2 m real | 6.63 s | 2.53 s | 2.18 s | 1.2× | 3.0× | 2.03 s |
| 2 m synthetic | 53.7 s | 16.3 s | 11.3 s | 1.4× | 4.7× | 9.3 s |
| 1 m synthetic | 208.4 s | 60.3 s | **43.8 s** | 1.4× | **4.75**× | 33.9 s |

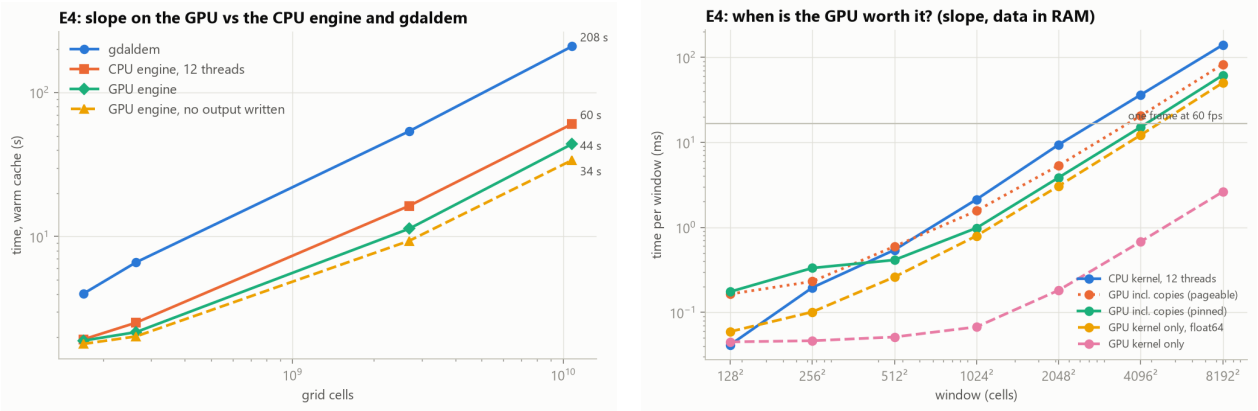



**Figure 11:** Left: GPU vs CPU vs gdaldem. Right: one window in memory: kernel alone vs kernel with copies vs CPU.

- **Kernel 50× faster, run 1.0–1.4× faster.** The GPU kernel needs 1.1 s for the whole 1 m grid (CPU engine's kernel: 34 s).
- **Copying is 95% of the GPU's time**: for a $2048^2$ window, kernel 0.18 ms, kernel + copies 3.85 ms.
- **Crossover:** the GPU (with copies) beats the CPU from $512^2$ cells for slope ($256^2$ for three products); below that a round trip (≥0.17 ms) loses. From $1024^2$ up: 2.2–2.4× (slope), 3.6–3.7× (3 products).
- **Fused products gain the most**: 81 s at 1 m vs 135 s CPU (1.7×); 3.1 s vs 18.5 s for three gdaldem runs on 2 m real (5.9×).
- **How the GPU is used matters more than the GPU** (2 m synthetic): straightforward CuPy port 17.0 s (*slower* than the CPU engine, 16.3 s) → own kernel 12.6 s → pinned buffers 11.1 s → 2 streams 11.3 s. The GPU part halves at each step (14.5 → 7.6 → 3.7 → 2.0 s) but the total stops improving.
- **float64** makes the kernel 8–24× slower but the total almost unchanged (11.5 vs 11.3 s).
- **GPU memory never the limit**: at most 2.0 GiB window by window; a whole grid at once would need 22 GB (2 m synthetic) or 86 GB (1 m).

**The bottleneck moves:** at 1 m the GPU engine waits 25.7 s for input and spends 7.8 s on the GPU. With 8 readers the read wait falls to 3.2 s, but then the writer is the limit (30.6 s wait, 43 GB at ~1.1 GB/s): total only 43.8 → 42.6 s. Computation → decompression → writing.

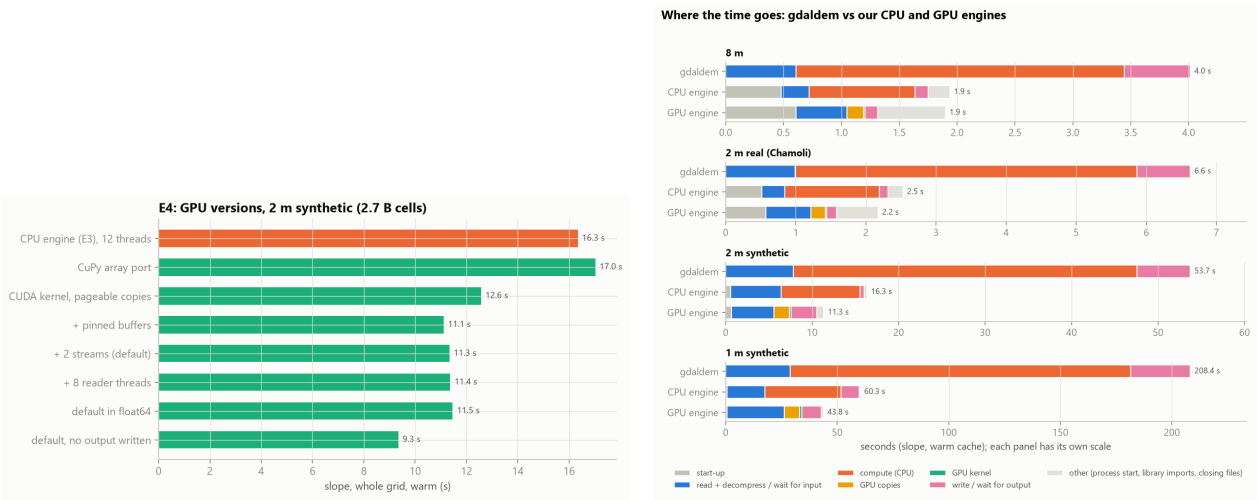



**Figure 12:** Left: GPU optimisation steps (2 m synthetic). Right: where the time goes, gdaldem vs CPU vs GPU engine.

**Key idea.** Phase 3 answer: parallel computation removes gdaldem's bottleneck (2–3.5×, up to 4.75× with the GPU), and the run becomes limited by moving data: decompressing input, copying over PCIe, writing output. Faster kernels alone cannot help any more; reading and writing less can, which is exactly what on-demand display does.

# 8 Phase 4 results: derivatives on demand from the pyramid (G2)

## 8.1 Accuracy

**Table 5:** Slope from each pyramid level vs the full-resolution truth (fully valid cells).

| DEM | Level | Cells scored | RMSE | Bias | >5° | Shown / true mean | RMSE exact pyramid |
|---|---|---|---|---|---|---|---|
| Chamoli 2 m | 4 m | 61.4 M | **2.3°** | −0.5° | 4.1% | 0.99 | 2.3° |
| | 8 m | 15.2 M | **4.4°** | −1.2° | 15.5% | 0.97 | 4.4° |
| | 16 m | 3.7 M | **5.6°** | −2.2° | 23.4% | 0.94 | 5.6° |
| | 32 m | 0.91 M | **6.4°** | −3.3° | 30.5% | 0.91 | 6.4° |
| | 64 m | 0.22 M | **7.9°** | −5.0° | 43.1% | 0.87 | 7.8° |
| | 128 m | 51 k | **10.0°** | −7.3° | 57.8% | 0.81 | 9.9° |
| HMA 8 m | 16 m | 29.6 M | **2.6°** | −0.7° | 6.6% | 0.98 | 2.4° |
| | 32 m | 6.5 M | **4.0°** | −1.6° | 15.6% | 0.95 | 3.9° |
| | 64 m | 1.3 M | **5.3°** | −2.9° | 26.3% | 0.90 | 5.3° |
| | 128 m | 0.23 M | **6.9°** | −4.5° | 40.5% | 0.83 | 6.9° |
| | 256 m | 29 k | **9.0°** | −6.8° | 58.7% | 0.73 | 8.8° |

- **Why too low:** averaging the DEM smooths the terrain before slope is computed, which removes steepness. Known since Zhang & Montgomery (1994) for coarser grids; new here is the measurement on pyramid levels against the time saved.
- **Levels up, not metres:** both DEMs give similar errors for the same number of levels up. Rule of thumb: ∼2.5° RMSE one level up, then on average ∼1.5° more per further level.
- **Steep ground loses most:** at 32 m the bias is −0.9° on 0–10° ground, −4.0° on 40–50°. The share of terrain ≥45° falls from 29.3% (truth) to 21.9% at 32 m and from 27.4% to 10.1% at 128 m: thresholded maps (landslide susceptibility) must not use coarse levels.

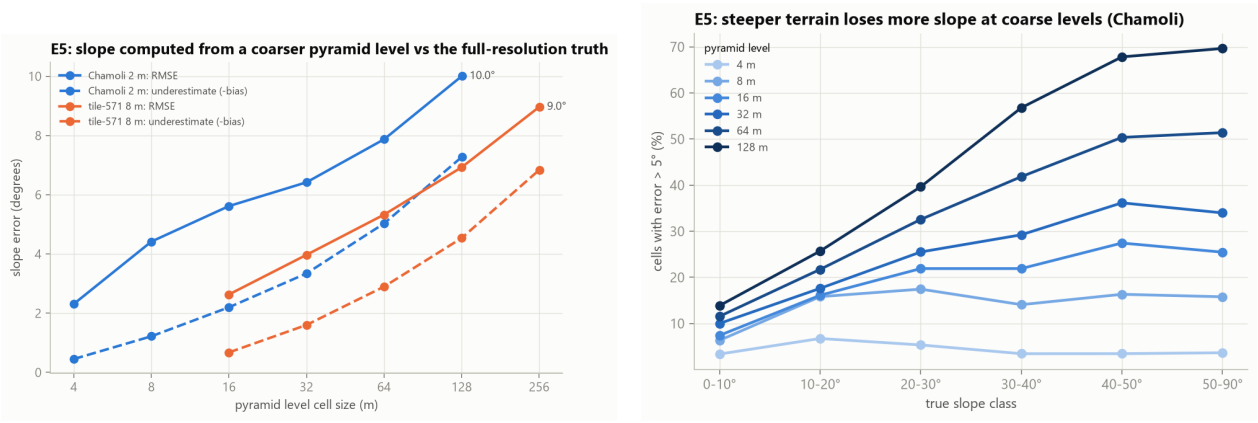



**Figure 13:** Left: RMSE and underestimate per level. Right: share of cells off by >5° by true slope class (Chamoli).

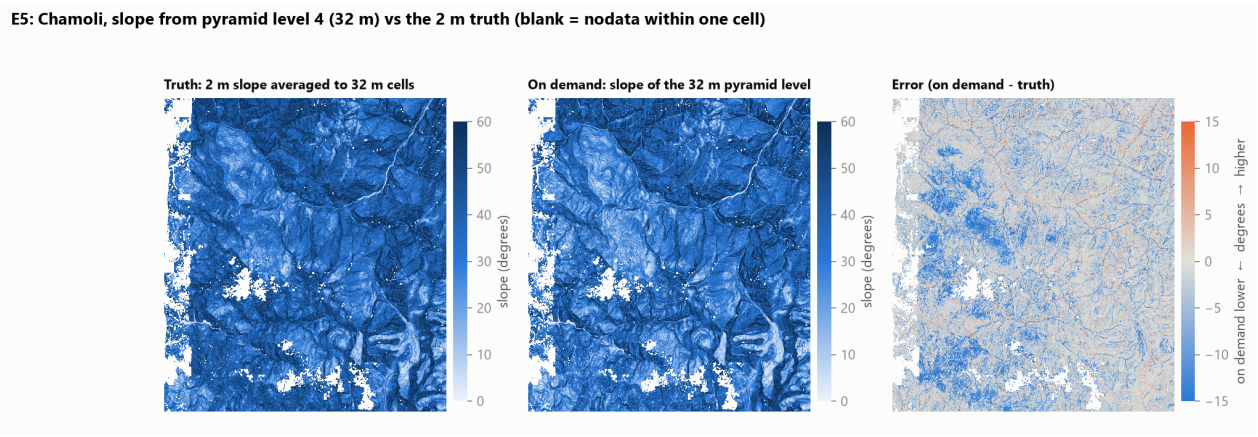



**Figure 14:** Chamoli at 32 m: truth, on demand, error (blue = on demand lower).

- **Aspect survives:** median error 1.8° (4 m) to 8.5° (128 m); only 1.4–13.7% of cells off by more than 22.5° (one 16-point compass sector).
- **Pyramid of good data beats another sensor:** 2 m averaged to 8 m: 4.5° vs HMA 8 m 8.6°; to 30 m: 6.3° vs Copernicus 30 m 9.4°; to 90 m: 8.7° vs 9.9°.
- **GDAL's overviews behave like an exact pyramid** (within 0.2°) despite the rounding.

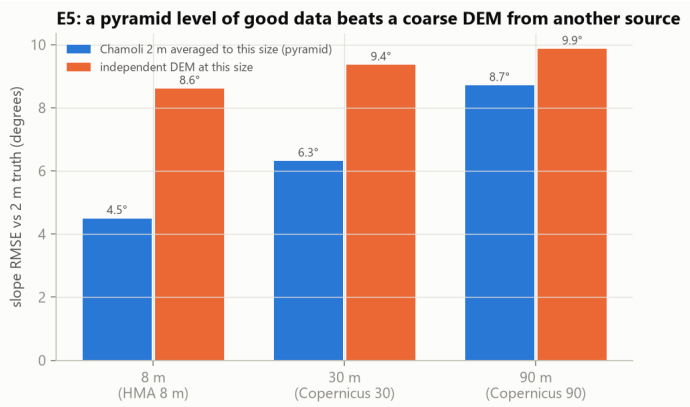



**Figure 15:** A pyramid level of the 2 m data vs an independent DEM of the same resolution.

### 8.2 Time

**Table 6:** One $512^2$ display tile, median of 8 places (ms, warm / cold), and time saved by on demand (warm).

| DEM | Level | On demand | Full res + average | Precomputed | Time saved |
|---|---|---|---|---|---|
| Chamoli | 4 m | 7.2 / 7.6 | 56 / 55 | 5.0 / 6.1 | 7.8× |
| | 8 m | 6.4 / 7.3 | 192 / 194 | 4.3 / 5.3 | 30× |
| | 16 m | 6.6 / 7.3 | 624 / 626 | 3.8 / 4.9 | 94× |
| | 32 m | 6.4 / 6.8 | 2867 / 2823 | 3.8 / 4.8 | **447**× |
| HMA 8 m | 16 m | 6.9 / 7.1 | 52 / 57 | 4.2 / 5.3 | 7.6× |
| | 128 m | 6.2 / 7.0 | 1186 / 1204 | 3.2 / 4.1 | 193× |

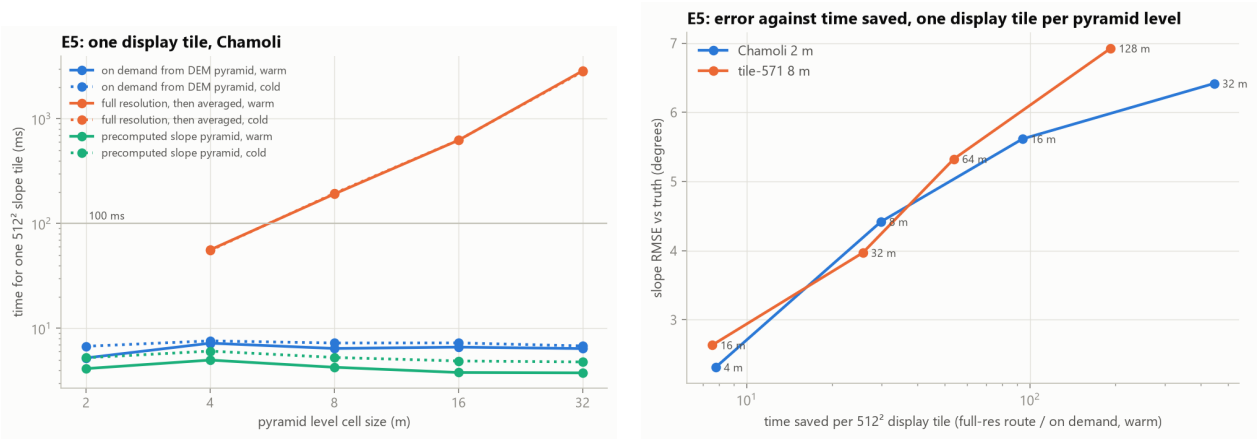



**Figure 16:** Left: tile time per level, three routes. Right: the G2 trade-off, RMSE vs time saved.

- **On demand is flat** (5–8 ms per tile at every level): a tile always reads $514^2$ cells. The exact route reads 4× more ground per level (2.9 s per tile at 32 m). Whole Chamoli area at 32 m: 13 ms vs 4.7 s.
- **Precomputed** is 1–3 ms faster per tile and exact, but costs a full pass and storage per derivative: Chamoli slope pyramid 10.6 s and 1.0 GB (more than the 650 MB DEM pyramid); all ten configurations ∼10 GB, rebuilt for every parameter change. On demand needs none.
- Cold vs warm: at most 1.5 ms per tile (on demand).

**Key idea.** Phase 4 answer to G2: each pyramid level up saves 2–5× time per view (8–450× per tile for 1–4 levels) and costs ∼2.5° slope RMSE for the first level and ∼1.5° per further level, as a systematic underestimate, worst on steep ground. Aspect costs much less. Good for display; not for slope thresholds.

## 9 Phase 5: the interactive viewer and E6

### 9.1 What changed and how the work was organised

Our earlier plan was a *box-zoom* viewer: see the whole map, drag a box, compute only that box, repeat. The professor asked for something closer to a real map: **pan and zoom continuously**, compute only what is in view at the resolution the screen needs, and **prefetch** what the user will probably look at next. Kartikeya turned this into an engineering spec (`docs/PHASE5_DEBOUNCED_ZOOM_SPEC.md`); `docs/FINAL_PLAN.md` is the working plan we followed. It adopts the spec where it is the best choice and differs where noted. **Careful with the numbering:** the final plan calls everything up to our Phase 4 "Phase 0" and splits our Phase 5 into its Phases 1–5:

| Final plan | This report | What | Owner |
|---|---|---|---|
| Phase 0 | Phases 0–4 | everything in Sections 1–8 | done |
| Phase 1 | §9.3 | tile engine core (backend only) | Swayam |
| Phase 2 | §9.4 | scheduler and server API | Swayam |
| Phase 3 | §9.5 | web viewer | Kartikeya |
| Phase 4 | §9.6 | QGIS plugin | shared |
| Phase 5 | §9.7–9.9 | E6 benchmark, tuning, results | Siddharth |
| Phase 6 | §13 | final report, slides, demo | shared |

**What Phases 0–4 told the design** (from measurements, not assumptions):

- A $512^2$ tile costs 5–8 ms on demand, of which the slope kernel is ∼0.5 ms. **Decoding, not computing, is the cost.**
- Every pyramid level is stored in $512 \times 512$ compressed blocks. A $512^2$ window with a 1-cell halo decodes up to **9 blocks**; a shared cache of decoded blocks can bring that to about 1 per tile.
- 12 performance-core threads beat all 22; efficiency cores are 1.7× slower and slow the foreground when mixed in.
- Exact (full-resolution) tiles cost up to 2.9 s at 32 m: never prefetch them.

**The goals (from the spec).** (1) Continuous pan and zoom over a fixed $512^2$ tile lattice per level, box zoom kept as Shift + drag and `POST /view`. (2) Debounced zoom: commit once after the last wheel tick; during the gesture show existing tiles rescaled, never a blank area. (3) Speculative prefetch of the 8 neighbouring views at low priority on the efficiency cores, cancelled when the user moves elsewhere. (4) Caches at every tier. (5) Cancellation by generation. (6) Bounded memory (server ≈2.2 GB budget). (7) Exact mode without prefetch; the expected E5 error of the shown level displayed. (8) E6 with two gates: **gate 1** prefetch raises foreground p95 by at most 10%; **gate 2** the debounce cuts server tiles per zoom gesture while the first frame arrives <100 ms after the debounce.

### 9.2 Architecture

A visible tile travels top to bottom in Figure 17: the browser asks for it with its session, generation and priority; the server answers from the result cache or submits a job to the scheduler; a worker renders it from cached or freshly decoded blocks; the PNG goes back with its timings and is kept in the result cache.

### 9.3 Tile engine core (final plan Phase 1)

Files: `demlab/view.py`, `kernels/tile_k.py`, `blockcache.py`, `resultcache.py`, `slru.py`, `colormap.py`, `cancel.py`. Design and why:

- $512^2$ **lattice aligned to the COG blocks.** Every level is stored in 512 blocks (checked), so one tile = one stored block plus its 1-cell halo. Each level's lattice comes from its own geotransform and size (GDAL's levels are not exact halvings, Phase 4). Edge and partial tiles are filled with nodata (transparent); tiles outside the DEM are answered as empty.
- **Level choice:** the finest level whose cell is ≤ one device pixel, using real cell sizes (not $2^k$), with ±10% hysteresis. The browser mirrors the same rule (tested on a shared fixture).
- **GIL-free per-tile kernels.** One tile is rendered on one thread by separate single-threaded `nogil` Numba kernels (slope / aspect / hillshade, TRI, TPI, roughness), so many tiles run in parallel on many threads. The spec suggested recompiling the existing kernels; we added

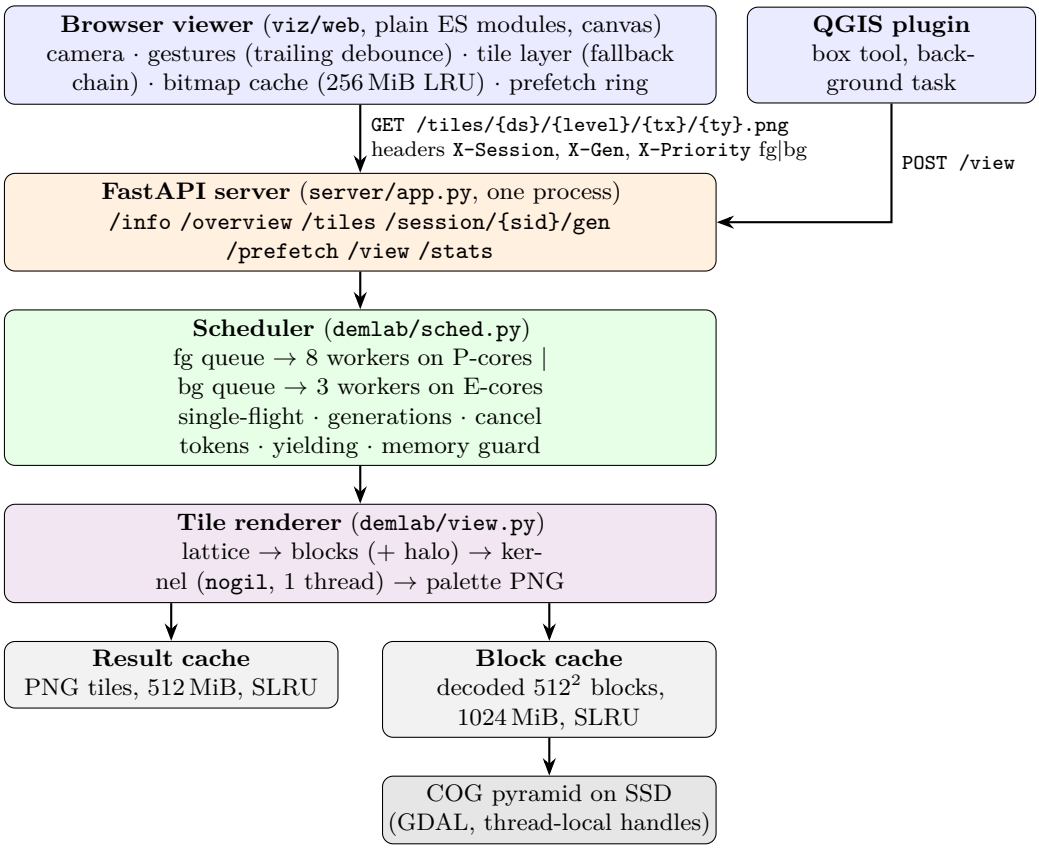



**Figure 17:** The Phase 5 system. One server serves both the browser and QGIS, so they share the caches. The policy numbers (debounce, prefetch) live in `viz/web/policy.json`, read by the browser and by the E6 virtual client.

separate ones so the parallel engine kernels stay untouched and E3/E4 stay reproducible. One Numba and one GDAL thread per tile; fixed pools (no oversubscription).

- **Block cache.** Our reader reads whole aligned blocks, decodes each once (single-flight), keeps them as float32 in a byte-budgeted SLRU (1024 MiB) and assembles windows from them. All-nodata blocks are a 0-byte marker. This removes the halo re-decoding found in E3.
- **Result cache + single-flight.** Finished PNGs (512 MiB, SLRU), keyed by a BLAKE2b hash of the canonical tile specification (dataset, level, tile, operator and every parameter); 100 identical concurrent requests compute once.
- **Palette PNG.** The colour map has 256 entries, so a tile is one byte per cell plus a palette (index 0 transparent = nodata), smaller and faster to encode than RGBA. The quantiser is a `nogil` Numba function and the PNG is written by our own small writer with `zlib` in run-length mode (Z_RLE): 2–3× faster than default deflate for 5–15% larger tiles. Both release the GIL.
- **Modes:** *auto* (the level's own cells, E5 error), *exact* (full resolution, then averaged, as E5's truth), *precomputed* (E5's stored slope pyramid). Per-stage timings (read, decode, kernel, colour, encode) for every tile.

**Gates and results.** 24 new tests (143 in total). Every auto-mode tile equals the existing engine's result on the whole level, cell for cell, for all 8 operator configurations, on every level of a test pyramid with data gaps and partial edge tiles. **Parallel gate** (`bench/check_tile_parallel`, Chamoli 2 m, 64 tiles, warm blocks): 8 threads **5.4**× faster than 1 (8.2 ms → 1.5 ms per tile of throughput; gate ≥5×); with NumPy quantising and Pillow encoding it was 4.3×. **Block cache:** 64 neighbouring tiles decode **100 blocks instead of up to 576**; warm tiles decode none.

### 9.4 Scheduler and server (final plan Phase 2)

Files: `demlab/sched.py`, `service.py`, `cpus.py`, `server/app.py` (`python -m server`). Design and why:

- **Two pools pinned to core classes.** Foreground (visible) jobs go to 8 workers pinned to the 12 performance-core threads (CPUs 0, 1, 10–19); background (prefetch) jobs to 3 workers on the 8 efficiency cores (CPUs 2–9). They never share workers, because E3 showed that efficiency cores mixed in slow the foreground. Pinning is per thread (`SetThreadAffinityMask`); the low-power cores (20–21) are not used.
- **Queue order:** newest generation first, then distance from the view centre / pan direction.
- **Single-flight with tickets.** Each request holds a ticket on a job; identical requests share the job. A foreground request for a tile queued as prefetch *promotes* it. A job is cancelled only when its last live ticket is gone.
- **Generations.** Every zoom commit raises the generation of the viewer's session, announced with `POST /session/{sid}/gen`; older work of that session is superseded (HTTP 409).
- **Cancel tokens**, checked before dequeue, after each block read, before the kernel and before encoding. GDAL reads cannot be interrupted, so the worst case is one block decode. Exact-mode tiles are computed in strips one stored block row deep and stop at the next strip.
- **Prefetch yields:** a background worker does not start a new tile while any foreground tile is queued or running (a running one is not interrupted).
- **Backpressure and memory guard.** The prefetch queue holds at most 256 jobs (oldest generation, then farthest, dropped first). Prefetch pauses below 1.0 GB free RAM, and paused prefetch requests get HTTP 503 at once.
- **Event-driven disconnect detection:** a closed connection (a browser abort) cancels the job (HTTP 499 internally).
- **API:** `GET /info` (datasets, levels, expected E5 error, legends), `GET /overview` (coarsest level, the base layer), `GET /tiles/{dataset}/{level}/{tx}/{ty}.png` (ETag, cache headers, `Server-Timing`, `X-Expected-Error`), `POST /prefetch`, `POST /view` (a box as PNG or GeoTIFF, rendered from the shared block cache), `GET /stats`, and the viewer itself at `/viewer/`. Settings by environment variables (cache budgets, worker counts, guard, yielding).

**Gates and results** (17 new tests, 160 in total; `bench/check_server`, real server, Chamoli 2 m):

- **Isolation:** foreground p95 16.6 ms idle vs 16.6 ms with 256 prefetch tiles queued (**+0.4%**; median of 3 rounds × 100 tiles; gate ≤+10%). Before yielding it swung between +6% and +15%.
- **Cancellation:** an aborted request still queued is dropped in a median **37 ms** (gate 50 ms). A running exact-mode tile stops at its next strip (7–155 ms).
- **Core classes** detected per thread as above; **memory guard** works (with 1.9 GB free it paused prefetch as designed).
- **Deviation** from the spec's URL: tiles carry the dataset name, because two datasets are served.

### 9.5 Web viewer (final plan Phase 3)

`viz/web/` (plain ES modules, no build step), served by the tile server: `python -m server`, then open `http://127.0.0.1:8765/viewer/`. Design and why:

- **Camera:** UTM centre and metres per device pixel; device-pixel ratio capped at 1 for level choice (HiDPI screens would multiply the work).

- **Tile layer:** every frame draws the needed level's tiles; a missing tile falls back to a coarser cached level, then a finer cached level, then the overview (always loaded), so no cell is ever blank. During gestures the preview is the browser's scaled `drawImage` of the fallback tiles.
- **Bitmap cache:** byte-accounted LRU of 256 MiB, other levels evicted first, `close()` on evict.
- **Gestures:** wheel (cursor-anchored), pinch, drag, keyboard. **Zoom commits once after a trailing debounce** (150 ms after E6); **pan requests new tiles at once** (no debounce: a pan does not change the level). Foreground tiles are requested from the centre out; every request carries the session's generation and is aborted (`AbortController`) when no longer visible.
- **Prefetch ring:** after a commit, the 8 view-sized rectangles around the view at the committed level, velocity-first, at most 2 in flight (leaving 4 of the browser's 6 connections for the foreground), recentred at a 25% offset (at most once per 100 ms), shrunk to the 4 cardinals above 150 tiles. Off in exact mode, in a hidden tab, and for 2 s after a slow foreground (p95 of the last 20 tiles >250 ms) or a 503. Requests time out after 10 s.
- **UI:** operator, method, mode (auto / exact / precomputed), legend, level, cell size and expected error, breadcrumb, Back, Whole map, and Shift + drag box zoom.
- **One policy file** (`viz/web/policy.json`) read by the browser and by the E6 virtual client.

**Checks.** 6 Node unit tests (level choice equals the server's on a fixture; lattice, camera, cache eviction, prefetch ring). In-browser self-test (`/viewer/?selftest=1`) in headless Chrome against the real server (Chamoli 2 m, hillshade), all four pass:

| Check | Result |
|---|---|
| 2 s wheel burst (120 ticks at 60 Hz) | 0 commits during the burst, exactly 1 after the debounce (level 4 → 2); in real-time Chrome the first exact frame comes 88 ms and the full view 161 ms after the commit |
| Prefetch ring after the commit | 52 neighbour tiles loaded at background priority |
| Pan by half a view at the settled level | 0 commits; all 8 new tiles from the prefetch ring, no foreground requests |
| Shift + drag box zoom on the central quarter | 1 commit, 4× zoom, level 0 |

### 9.6 QGIS plugin (final plan Phase 4)

`qgis_plugin/geoguessers_view` ("GeoGuessers terrain"; install with `qgis_plugin\install.ps1`, notes in `qgis_plugin/README.md`). A panel (server, dataset, product, method, values auto / exact / precomputed, cell size match-the-box-on-screen / full resolution, sun direction) and a box tool. **How it works:** the user drags a rectangle; the plugin reprojects it to UTM 44N (the map can use any CRS), sends it with the map's screen width to `POST /view` in a background `QgsTask` (QGIS stays usable), and adds the returned GeoTIFF as a styled layer, with level, cell size and the E5 expected error in its metadata. The server picks the level whose cells match the box on screen, so a large box comes back quickly from a coarse level, and a smaller box inside it at a finer level. Same server, same caches as the browser. A full-resolution box is limited to 64 M cells (about 16 km × 16 km at 2 m; HTTP 400 with the reason), so one request cannot build a multi-GB mosaic.

**Check** (`qgis_plugin/check_plugin.py`, headless, QGIS 3.44's Python against the real server): 13/13 pass. A full-resolution slope box lands on the DEM's cell grid and is **identical** to gdaldem's slope of the full-resolution DEM (1.22 M cells, max difference 0); a box drawn on a WGS 84 map gets exactly the level the rule gives; both datasets work; an oversized box and a box outside the data give messages instead of layers. Still to do by hand: one look in the QGIS GUI.

## 9.7 E6 method (final plan Phase 5)

**Virtual client.** `bench/vclient.py` ports the viewer to Python: camera, level choice with hysteresis, byte-budgeted bitmap cache, trailing debounce, generations, aborts, prefetch ring and back-off. It reads the same `policy.json` and replays a trace in real time against the live server: HTTP/1.1 through `httpx` with at most 6 connections (a browser's limit), 60 Hz frames, and an abort closes the connection as `AbortController` does. It runs its own event loop on the high-resolution clock (Problems, Section 11).

**Traces** (`bench/traces/*.json`, seeded, 1600×900 view; 17 in total):

| Family | Traces |
|---|---|
| `pan-line` | 5 s at 0.5, 1 and 2 view widths per second, roughly diagonal, at level 0 |
| `pan-reverse` | the same, turning back at 2.5 s |
| `pan-walk` | 6 s random walk, new heading every 0.3–1 s |
| `zoom-burst` | 60 Hz wheel ticks for 0.3 / 1 / 2 s, in and out, 1 / 2 / 3 levels (6 traces) |
| `zoom-notched` | 10 discrete notches 30–150 ms apart (brisk) or 120–300 ms apart (slow), 2 levels in |
| `zoom-pan` | 10 s of alternating 0.5 s bursts (one level) and 1 s pans |
| `box` | three nested box zooms (×4 each) from the whole map to full resolution |

**Protocol.** One factor at a time around the defaults, 24 configurations (debounce off / 100–250 ms / forced commit; prefetch off, back-off variants, fixed ring order; background shared on P-cores, no yielding, 2 or 4 bg workers; 4 or 6 fg workers; block cache 0 / 256 MiB; result cache off; slope, aspect, precomputed; 8 m dataset; cold OS cache). Every (configuration, trace): 1 warm-up + 5 measured runs; 996 runs (147 min) plus 72 for the notched traces = **1,068**. Each run starts a new server process (empty caches; kernels warmed on the other dataset), loads the overview and the first view with its prefetch ring, then starts the trace clock. Server counters from `/stats` at trace start and end; RSS and free memory every 50 ms; per-CPU times bracket the trace. Data: Chamoli 2 m (levels 0–6), hillshade, unless stated. The prefetch memory guard was set to 0.5 GB for E6 (the editor leaves the laptop with about 2 GB free); no prefetch request was refused in any run (free memory never below 633 MB). **The sweep's base debounce was 200 ms**; 150 ms became the default only after the sweep.

**Metrics.** Pan hit rate, blank area-time, TTFF, complete, server tiles per gesture, prefetch precision, and foreground p95 both at the client (request to response, including the wait for a connection) and on the server (queue + render, from `Server-Timing`); definitions in the glossary.

## 9.8 E6 results

**Table 7:** Default policy on every trace (median of 5 runs; debounce 200 ms, the sweep's base). TTFF and Complete (ms) for zoom traces include the debounce; Tiles/gest. = server foreground tiles per zoom gesture.

| Trace | Fg req. | Prefetched | Hit rate | TTFF | Complete | Tiles/gest. |
|---|---|---|---|---|---|---|
| pan-line 0.5/1/2 | 0 / 0 / 2 | 175 / 313 / 333 | 1.00 / 1.00 / 0.99 | | | |
| pan-reverse 0.5/1/2 | 0 / 0 / 5 | 73 / 128 / 443 | 1.00 / 1.00 / 0.98 | | | |
| pan-walk | 0 | 290 | 1.00 | | | |
| zoom in 0.3/1/2 s | 20 / 15 / 20 | | | 272 / 273 / 289 | 325 / 325 / 358 | 20 / 15 / 20 |
| zoom out 0.3/1/2 s | 18 / 20 / 12 | | | 269 / 264 / 257 | 321 / 314 / 276 | 18 / 20 / 12 |
| zoom-pan | 36 | 128 | 1.00 | 206 | 206 | 7.2 |
| box | 35 | 125 | | 64 | 127 | |

**On-the-fly computation keeps up** (Table 7):

- **Pans:** 98–100% of the tiles entering the view are already in the browser, at 0.5, 1 and 2 view widths per second, straight or reversing; blank area-time rounds to 0.000.
- **Zoom bursts:** the first exact tile appears 57–89 ms after the commit, the whole view 76–158 ms after it (the table's TTFF adds the 200 ms debounce).
- **Box zoom:** first exact tile 64 ms after the animation ends, whole view after 127 ms.
- **Cost:** the server never needed more than 982 MiB; prefetch used about 3 CPU-seconds of efficiency cores per 5 s pan.
- In zoom-pan, three of the five gestures zoom back out to cached levels (no requests), so the median gesture completes when the debounce fires; the two zoom-ins complete about 300–320 ms after the last tick.

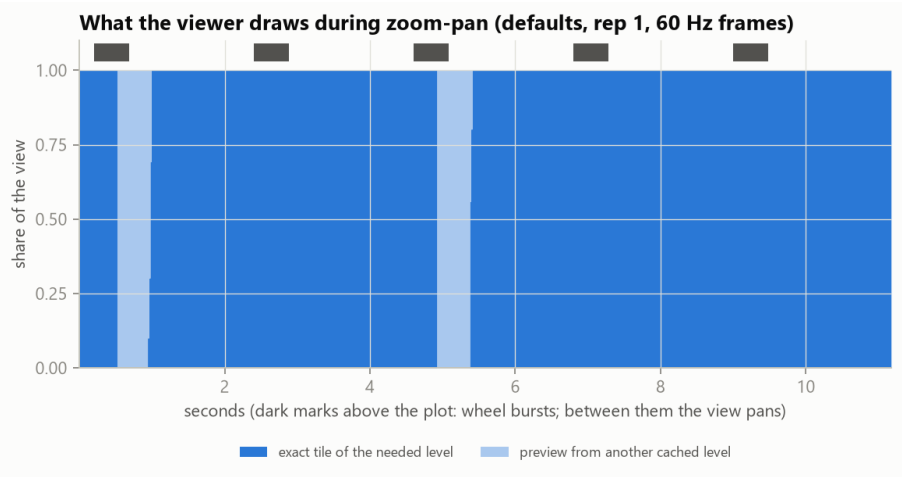



**Figure 18:** What the viewer draws during zoom-pan (defaults, 60 Hz frames): share of the view from the needed level's own tiles (dark) or from another cached level (light). Only after the two zoom-ins do coarser tiles stand in, for a few hundred milliseconds.

**Table 8:** Server foreground tiles per zoom gesture and delay, by debounce (median of runs).

| Debounce | 60 Hz bursts | Notched 30–150 ms | Notched 120–300 ms | TTFF − debounce, bursts (ms) | TTFF, bursts (ms) |
|---|---|---|---|---|---|
| off (every tick) | 62 | 36 | 48 | – | 6 |
| 100 ms | 19 | 36 | 48 | 63 | 163 |
| **150 ms (new default)** | 19 | **15** | **39** | 68 | 218 |
| 200 ms | 19 | 15 | 39 | 71 | 271 |
| 250 ms | 19 | 15 | 39 | 68 | 318 |
| 200 ms + forced commit at 500 ms | 35 | 30 | 39 | 62 | 262 |

**Gate 2: the debounce.**

- **Without a debounce** every tick commits (up to 121 commits in a 2 s burst): each changes the generation, aborts the previous commit's requests and starts the next. The server renders a median of **62** foreground tiles per gesture (it finishes most aborted tiles, which are short, and cancels 4–19 per burst mid-render).
- **With a trailing debounce** it renders **19 (3.3× fewer)** and the first exact tile arrives **63–71 ms after the debounce fires**, inside the 100 ms gate. **Gate 2 passes.**
- **What it trades:** a fixed wait after the last tick against wasted work. Committing on every tick buys only a sharper preview during the gesture (blank area-time 0.1 vs 0.4).

- **Why 150 ms:** at 60 Hz the ticks are 17 ms apart, so every value from 100 to 250 ms gives the same 19 tiles. A notched wheel decides it: with notches 30–150 ms apart, 100 ms commits between notches (36 tiles, as bad as no debounce), while 150, 200 and 250 ms coalesce the gesture (15). Slow scrolling (120–300 ms gaps): 48 at 100 ms, 39 at 150 ms and longer. 200 ms costs 40–50 ms more delay than 150 ms for no fewer tiles.
- **Forced commit** after 500 ms of continuous scrolling (`maxWaitMs`) almost doubles the tiles (35 vs 19) for 9 ms less delay: stays off.

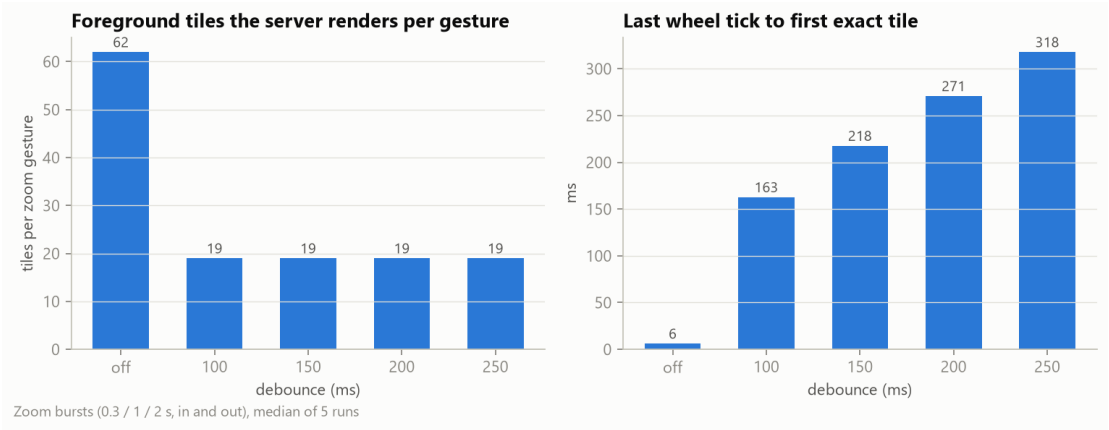



**Figure 19:** Zoom bursts at 60 Hz: foreground tiles the server renders per gesture, and time from the last tick to the first exact tile, against the debounce.

**Table 9:** Prefetch policies over the seven pan traces and zoom-pan (40 runs each; median [interquartile range]).

| Policy | Foreground requests | Pan hit rate | Prefetched | Precision |
|---|---|---|---|---|
| prefetch off | 104 [68–135] | 0.30 [0.00–0.51] | 0 | – |
| defaults (back-off if foreground p95 > 250 ms) | 0 [0–4] | 1.00 [1.00–1.00] | 218 | 0.34 |
| first policy (back-off at 100 ms) | 0 [0–56] | 1.00 [0.70–1.00] | 192 | 0.29 |
| no back-off | 0 [0–4] | 1.00 [1.00–1.00] | 232 | 0.34 |
| ring in fixed order | 0 [0–7] | 1.00 [0.97–1.00] | 292 | 0.33 |

**Prefetch.**

- **With prefetch** pans need 0–5 foreground requests instead of 36–202; on a straight pan the entering tiles are 99–100% cached. **Without** it 0% on a straight pan and about 50% on a pan that turns back over its own path. The view still looks almost complete without prefetch (blank area-time 0.2–3.4%): each new tile arrives in about 50 ms and the coarser cached level covers the gap.
- **Cost:** precision 0.20–0.56, i.e. 2–5 tiles fetched for each one used: a ring of 8 neighbours always loads directions the user does not take. The price is paid in efficiency-core time, not in foreground latency.
- **Ordering:** velocity-first keeps the hit rate at 0.99 at 2 view widths per second; a fixed order drops it to 0.91.
- **Back-off threshold:** at 100 ms (the first policy) prefetch switched off at random, because the cold-start p95 is about 100 ms; the fast reversing pan fell from 0.98 to 0.50 (19 back-offs per run). At 250 ms the policy equals "no back-off" on every trace and keeps a safety valve for a really overloaded server.

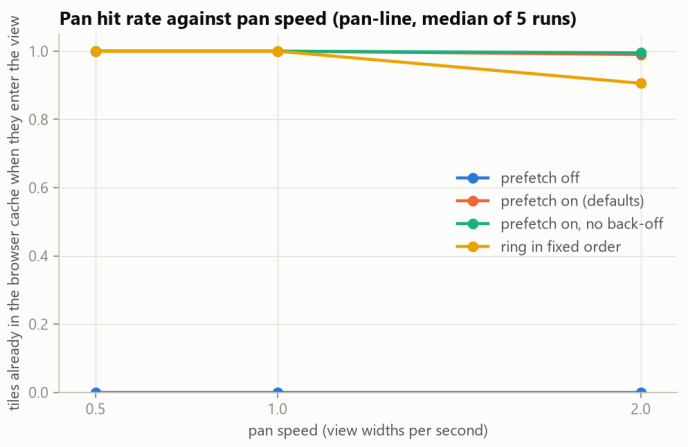



**Figure 20:** Pan hit rate (tiles already in the browser when they enter the view) against pan speed.

**Table 10:** Foreground p95 with and without prefetch. None of these foreground tiles overlapped a prefetch request.

| Trace | Configuration | Fg tiles rendered | Server p95 (ms) | Client p95 (ms) |
|---|---|---|---|---|
| box (same tiles) | prefetch off | 175 | 94.7 | 137.6 |
| box (same tiles) | prefetch on | 175 | 89.7 | 129.3 |
| zoom-pan | defaults: E-cores + yield | 180 | 58.1 | 103.5 |
| zoom-pan | E-cores, no yield | 180 | 63.6 | 111.6 |
| zoom-pan | P-cores (shared) + yield | 180 | 52.1 | 94.8 |

**Gate 1: prefetch and the foreground.**

- **Like for like:** on the box trace the foreground tiles are identical with prefetch on and off. Server p95 is **90 ms with prefetch vs 95 ms without** (client 129 vs 138 ms): prefetch is slightly *faster*, because it has already decoded shared blocks. **Gate 1 passes.**
- **Why (structural):** prefetch stops at the first input event of a gesture and restarts only when the view is complete. With the defaults only 25 of 905 foreground tiles (3%) were ever requested while a prefetch request was in flight. On top of that the server keeps prefetch on other cores and lets it yield.
- **Placement:** on zoom-pan, letting prefetch start while foreground work is pending raises server p95 by 9% (64 vs 58 ms). **Pinning held:** 20,242 foreground jobs ran on performance cores only and 122,215 prefetch jobs on efficiency cores only.
- **Why not the spec's form of the gate:** per-trace foreground p95 with prefetch on vs off compares *different tiles* (with prefetch on, pans need almost no foreground tiles; what remains are cold tiles at a new level after a zoom), so we used the like-for-like box comparison.




**Figure 21:** Foreground tile latency (p95) on identical tile sets: on the server (queue + render) and at the client (also waiting for one of six connections).

**Table 11:** Mixed traces (pan-walk, zoom-pan, two bursts, box; 25 runs each; medians).

| Configuration | Client p95 ms | Server p95 ms | Complete ms | Blocks decoded | Peak RSS MiB |
|---|---|---|---|---|---|
| defaults (8 workers, 1024 MiB block cache) | 104 | 98 | 260 | 185 | 608 |
| 4 / 6 foreground workers | 106 / 101 | 76 / 92 | 254 / 251 | 185 | 588 / 598 |
| no block cache | 146 | 111 | 285 | 1,128 | 386 |
| 256 MiB block cache | 108 | 94 | 260 | 185 | 608 |
| result cache off | 103 | 91 | 257 | 185 | 576 |
| slope / aspect | 115 / 115 | 97 / 103 | 255 / 258 | 185 | 606 / 608 |
| precomputed slope | 93 | 56 | 248 | 160 | 562 |
| HMA 8 m DEM | 118 | 92 | 268 | 130 | 594 |
| OS file cache flushed | 103 | 104 | 257 | 185 | 608 |

**Workers, caches and the rest.**

- **The browser's 6 connections, not the server, set the foreground latency.** Client p95 is about 40 ms above server p95 (box: 90 → 129 ms) because tiles queue for a connection. 4, 6 or 8 foreground workers give the same client p95 (101–106 ms); with 4 the server p95 is even lower (76 vs 98 ms, less contention for decoding), but one browser cannot use more than 6 at once. 8 stay the default so a second client (QGIS, another tab) does not queue.
- **Block cache:** without it the server decodes 6× more blocks (1,128 vs 185) and client p95 rises 40% (146 vs 104 ms). 256 MiB equals 1024 MiB over these 1–11 s traces because the cache never fills.
- **Result cache:** no effect here, since each run uses a fresh server and the browser keeps its own tiles, so no tile is asked for twice. It matters across sessions and clients, which E6 does not exercise.
- **Product, mode, data, OS cache:** slope and aspect tiles are about 10% slower than hillshade; precomputed slope is the fastest on the server (p95 56 vs 98 ms: one block, no halo, no kernel); the 8 m dataset behaves like the 2 m one; a flushed OS file cache changes nothing, because decoding, not the disk, is the cost (as in E1–E3).

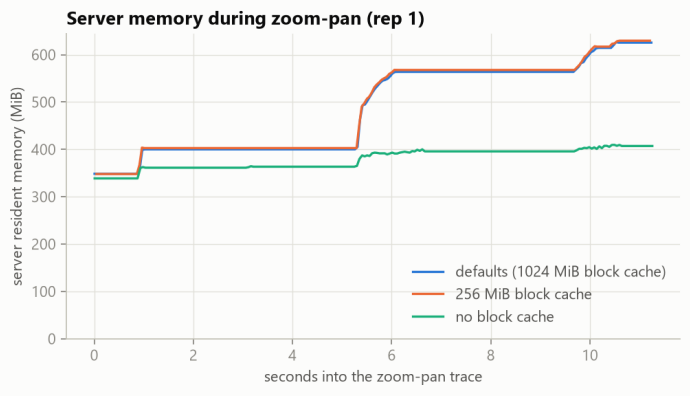



**Figure 22:** Server memory during zoom-pan (one run): steps when a zoom-in decodes a new level's blocks; without the block cache memory stays lower but every block is decoded again.

**A real browser.** `bench/e6_browser.py` replays three traces in headless Chrome, driven in real time over the DevTools protocol, and compares with the virtual client:

| Trace | Fg requests (Chrome / virtual) | Commits | TTFF ms (Chrome / virtual) | Complete ms (Chrome / virtual) |
|---|---|---|---|---|
| zoom-burst-in-1 | 15 / 15 | 1 / 1 | 248 / 273 | 298 / 325 |
| pan-line-1 | 0 / 0 | 0 / 0 | – | – |
| box | 35 / 35 | 3 / 3 | 46 / 64 | 123 / 127 |

Same commits, same foreground requests, the same prefetched tiles (120 vs 120; 306–310 vs 308–313), view-complete within 10%, first frames 18–25 ms sooner in Chrome. The Python client is slightly conservative, so E6 latencies are upper bounds.

## 9.9 Policy set from the data

**Table 12:** Defaults in `viz/web/policy.json` and on the server after E6.

| Setting | Before E6 | Now | Why |
|---|---|---|---|
| `debounceMs` | 200 | **150** | Same tiles as 200 and 250 ms on every trace, notched wheels included; 40–50 ms sooner; 100 ms commits between notches |
| `maxWaitMs` | off | off | A forced commit nearly doubles tiles per gesture for 9 ms |
| `prefetch.backoffP95Ms` | 100 | **250** | 100 ms sits at the cold-start p95 and switches prefetch off at random; 250 ms equals "no back-off" everywhere and keeps a safety valve |
| `prefetch.order` | velocity-first | velocity-first | 0.99 vs 0.91 hit rate at 2 view widths per second |
| `prefetch.maxInFlight` | 2 | 2 | Leaves 4 of the browser's 6 connections for the foreground; hit rate already 0.98–1.00 |
| Server fg workers | 8 | 8 | 4–8 equal for one browser; 8 leave room for a second client |
| Server bg placement | E-cores + yield | E-cores + yield | Verified on every job; not yielding costs 9% foreground p95 |
| Block cache | 1024 MiB | 1024 MiB | 256 MiB is enough for short traces; longer sessions and QGIS reuse more blocks |

Other values (unchanged by E6): level hysteresis 10%, device-pixel-ratio cap 1, browser cache 256 MiB, ring recentre at 25% (at most every 100 ms), ring at most 150 tiles, back-off window 20 tiles

and 2 s, prefetch request time-out 10 s; on the server 3 bg workers, result cache 512 MiB, prefetch queue cap 256, and the memory guard at 1.0 GB free (lowered from 2.5 GB while building the viewer, because 2.5 GB kept prefetch paused on this laptop).

**Key idea.** Phase 5 answer: computing derivatives on demand keeps up with continuous pan and zoom on one laptop. A velocity-first ring of neighbouring views makes panning seamless at no cost to the foreground, because prefetch never runs while the user is waiting; a 150 ms trailing debounce cuts the server's work per zoom gesture 3.3×. The latency that remains is set by decoding compressed blocks and the browser's 6 connections, not by computing the derivatives: the same data-movement bottleneck Phases 2–4 found for whole grids.

## 10 The whole story (what to say in the presentation)

1. **Problem:** a 1 m DEM of our area is 43 GB, 2.6× the laptop's RAM.
2. **Standard tools** (gdaldem, QGIS) don't crash: they stream. But slope takes 3.5 min on one thread, and displaying without a pyramid takes 32 s per redraw. Bottleneck: single-threaded computation; for display, reading.
3. **Our engine** uses all cores (and optionally the GPU) in a bounded pipeline: 3.5× (CPU) to 4.75× (GPU) faster, identical results, constant memory. The naive "load it all" approach fails beyond ∼0.6 B cells.
4. **But** the bottleneck moves to data movement: decompression, PCIe copies, writing 43 GB of output. A GPU only pays when computation is a large share (fused products) or windows are large enough ($\geq 512^2$).
5. **So** don't process everything: read and compute only what is viewed, from the right pyramid level. That makes every view cost 5–8 ms regardless of zoom, with a known, measured accuracy cost (slope too low by ∼2.5° + 1.5°/level), and no storage for precomputed products.
6. **The viewer** turns this into a map: continuous pan and zoom over a fixed $512^2$ tile lattice, every tile computed on demand. A 150 ms trailing debounce cuts the server's work per zoom gesture 3.3×; a velocity-first ring of neighbouring views makes panning seamless (98–100% of new tiles already in the browser); prefetch runs on the efficiency cores and never while the user waits, so it does not slow the visible tiles.
7. **And the bottleneck is still data movement:** the viewer's remaining latency is decoding compressed blocks and waiting for one of the browser's 6 connections, not computing slope or hillshade. The same lesson at the scale of one tile.

## 11 Problems we faced and what we learned

| What went wrong | Lesson / fix |
|---|---|
| *Setup and data* | |
| Miniforge in the default path | Fails with a space in the user name; installed at `D:\miniforge3`. |
| First datasets too small / patchy | Switched to one study area with a real resolution ladder (HMA mosaic, Copernicus, Chamoli 2 m). |
| No free 1 m DEM | Synthetic cubic upsamples, timing only; tested that they time like real data but are useless for accuracy. |
| *Phase 2 benchmarking* | |
| `gdalinfo -checksum` as "read" | Did extra CPU work; "write" came out negative. A reference must do only what it measures. |

| What went wrong | Lesson / fix |
|---|---|
| Counters sampled every 50 ms | Missed the end of short runs; read exact totals from the finished process. |
| `RAMMap -Ec` | Not a real option; check flags before automating (`-Et`). |
| Unpinned runs | Hybrid CPUs need pinning; one run was 2× slow. |
| Defender + deleted 10–40 GB files | Deleted files are not freed while scanned: 108 GB filled the disk. Exclude only scratch folders; wait for freed space between runs. |
| Background activity (Chamoli) | "Foreign disk reads" are a cheap, reliable interference detector. |
| Moving small files to the external drive | Only >1 GB run outputs belong there; small files stay in the project; old ones in `old files/`. |
| Session temp scripts lost at restart | Copy every one-off script whose result we quote into `old files/` immediately. |
| *Phase 3 engine* | |
| All window reads submitted at once | Could hold ∼86 GB at 1 m; bounded prefetch (at most 6 windows in flight). |
| rasterio "boundless" read slow | Read the in-grid part and pad ourselves (8 m slope 1.55 → 1.12 s). |
| Writing in the compute thread | Blocked the kernel; one dedicated writer thread (GDAL: never write one dataset from several threads). |
| Numba used 22 threads when pinned to 12 | Oversubscription; set the thread count explicitly. |
| Mismatch with gdaldem on flat cells | gdaldem sums in float32; reproduced it in a parity mode to prove equality. |
| Small windows unexpectedly slow | The halo re-decodes neighbouring compressed tiles; 2048 windows. |
| D: drive has only ∼160 GB free | Outputs deleted after each run, max one run's output on disk. |
| GPU: straightforward CuPy port slower than CPU | Temporaries + float64; own CUDA kernel, pinned memory, streams. |
| Risk to the GPU / laptop | Temperature guard, memory cap, no driver or clock changes. |
| *Phase 4* | |
| GDAL overviews are not exact halvings | Footprints from geometry; exact-pyramid cross-check (difference ≤0.2°). |
| Data gaps would bias the error | Score only fully valid cells (cell + 8 neighbours). |
| Aspect undefined on flat / mixed blocks | Score where slope ≥5° and directions agree ($R \geq 0.5$). |
| *Audit (found by checking every number)* | |
| A table labelled "warm" mixed cold runs | Generator bug fixed; text built on it corrected. |
| "GB" that were MiB/1000 | One unit convention: MB/GB decimal, MiB only for GPU memory. |
| Overstated claims | e.g. "reads the whole 16.6 GB file" (really 12.7 GB), "sources differ more than the pyramid error" (false at 90 m), tile time "6–7 ms" (really 5–8). Write ranges from the data, not from memory. |
| Correctness gate on 90/30 m never run | Added; passes. Plan items need a checklist. |
| Median only, plan asked median + IQR | `run_stats` for every configuration. |

| What went wrong | Lesson / fix |
|---|---|
| *Phase 5: tile engine, scheduler and server (final plan Phases 1–2)* | |
| Parallel gate only 4.3× on 8 threads | NumPy colour quantising and Pillow PNG encoding held the GIL, so threads waited for each other. A `nogil` Numba quantiser and our own PNG writer with `zlib` (Z_RLE: 2–3× faster, 5–15% larger tiles): 5.4×. Check every stage of a threaded path for the GIL, not just the kernel. |
| Windows gives E- and LP-cores the same efficiency class | A split by class alone would mix the very slow LP-cores into the prefetch pool. We tell them apart by their separate last-level cache (CPUs 20–21) and leave them unused. |
| Cancelled job still running could be joined | A new request for the same tile could attach to a job that was cancelled but still finishing its stage, and that job's clean-up could remove the newer job from the table. Now a request never joins a cancelled job (a new one is created), and clean-up removes the entry only if it is still the same job. |
| Isolation gate noisy (+6 to +15%) | Even on separate cores, background threads share the GIL and memory bandwidth with the foreground. Prefetch now yields (no new prefetch tile while foreground work is pending): +0.4%. |
| Exact-mode tiles slow to cancel | A running tile can stop only between stages; exact tiles are now computed in strips one stored block row deep, so they stop at the next strip (7–155 ms). |
| Disconnects found by polling every 20 ms | Gave 58–80 ms (gate 50 ms). Event-driven disconnect detection: median 37 ms. |
| *Phase 5: web viewer and QGIS plugin (final plan Phases 3–4)* | |
| Memory guard at 2.5 GB kept prefetch paused | This laptop often runs with <2 GB free. Default lowered to 1.0 GB; paused prefetch now gets 503 at once, so the client backs off for 2 s instead of holding two connections open; prefetch requests time out after 10 s. |
| Prefetch never resumed in some cases | If prefetch could not start right after a commit (backing off, hidden tab), it did not start. Now it is marked pending and starts on a later frame. |
| Box zoom produced two commits | The animation's moves also started the debounce timer. Animated moves (box zoom, Back, Whole map) now suppress the timer and commit once at the end. |
| Blank map at start | The first view was fitted before the canvas size was known. The viewer now reads the canvas size before fitting the first view (and follows resizes). |
| Headless Chrome in virtual-time mode | It distorts every network wait. Switched to real-time headless Chrome driven over the DevTools protocol with the `websockets` package the server already uses: Playwright (which would have needed the team's OK to install) was not needed. |
| One QGIS box could build a multi-GB mosaic | `POST /view` refuses boxes over 64 M cells (HTTP 400 with the reason). |
| *Phase 5: E6 (final plan Phase 5)* | |
| Virtual client's timers up to 16 ms early | Python 3.12's Windows event-loop clock moves in 15.6 ms steps and fires timers up to one step early, so the debounce fired early. First sweep stopped after 17 runs (kept in `old files/results_superseded/`) and rerun on our own high-resolution loop (∼1 ms). |

| What went wrong | Lesson / fix |
|---|---|
| Back-off threshold of 100 ms | Sat at the cold-start p95 (∼100 ms: a fresh view's blocks decoding while 15–20 tiles queue on 6 connections), so prefetch switched off at random (pilot) and the fast reversing pan fell to a 0.50 hit rate. Raised to 250 ms before the sweep (the sweep still measures 100 ms as a variant). |
| The spec's gate 1 compares different tiles | Per-trace foreground p95 with prefetch on vs off: with prefetch on, pans need almost no foreground tiles, so the two sides are different tile sets. Used the like-for-like box trace (identical tiles). |
| Steady 60 Hz ticks hide the debounce choice | Gaps of 17 ms make 100–250 ms behave the same. Notched-wheel traces (30–150 and 120–300 ms gaps, 72 runs) added after the main sweep; they set 150 ms. |
| Laptop has only ∼2 GB free with the editor open | E6 sets the prefetch memory guard to 0.5 GB; no prefetch request was refused in any run (free memory never below 633 MB). |

## 12 Validity and limitations

- **One laptop.** All timings are for this machine (Core Ultra 9 185H, 16 GB, NVMe SSD, RTX 4060 Laptop, Windows 11). Ratios (speed-ups, stage shares) transfer better than absolute times.
- **Synthetic 2 m/1 m** grids add cells but no detail: timing only. Accuracy (Phase 4) uses real data only.
- **One region of steep terrain** (mean true slope 38° on Chamoli): flatter regions would show smaller pyramid errors; the per-class tables carry over.
- **Single warm runs** at 1 m (protocol choice, stated). Repeats agree closely (median IQR 1.2–1.6%).
- **Only GDAL/QGIS** as the baseline: other tools (WhiteboxTools, GRASS, xarray-spatial) were not benchmarked.
- **Truth is itself a DEM** with noise; the 8 m DEM's truth is only 8 m.
- **Python start-up** (∼0.5 s) is included in every engine time, like gdaldem's start-up: fair, but it dominates the small grids.
- **E6: one client on one laptop.** The result cache and the spare foreground workers are there for several clients (browser + QGIS); E6 runs one.
- **E6: short synthetic traces** (1–11 s), seeded rather than recorded from people. Caches never fill, so long sessions (evictions, the result cache) are not measured.
- **E6: simulated client.** The virtual client is a port of the viewer; the headless-Chrome check agrees on all counts and within 10% on view-complete times, but covers three traces.
- **E6: cold start only.** Every run starts with an empty server; a returning user would see faster tiles.
- **E6: CPU only.** The GPU path is not used for tiles: at $512 \times 512$ per tile, E4 predicts little gain once the transfer is counted (1.3× including the copies).
- **Not done:** the QGIS box-drawing baseline listed in the plan (QGIS's own rendering is covered by E1q), and one look at the plugin in the QGIS GUI by hand.

# 13 What is left

## 13.1 Final deliverables (final plan Phase 6)

- **Final report** (LaTeX, `report/final_report/`): introduction, literature, data, method, E1–E6 results, the bottleneck story (G1), the pyramid trade-off (G2), the viewer, limitations, conclusion.
- **Slides** (`report/slides/`) and a **recorded demo video** of the viewer and the QGIS plugin, as the fallback for the live demo.
- **Professor update covering Phases 3–5** (`docs/prof_update_final.md`).
- Update `README.md`, `PROJECT_PLAN.md` and this report; freeze the code, tag the release, push, refresh the copy on the external drive.

## 13.2 Stretch (only if the above is on track)

- GPU tile path as an E6 variant (expected small gain at $512^2$, per E4).
- Cesium globe showing the viewer's tiles draped on terrain.
- Viewshed (exact R3 vs approximate XDraw vs `gdal_viewshed`: time and share of cells classified differently, gap G3; heights first converted to one vertical datum) and terrain distance (Dijkstra vs A* on a slope-based cost surface): the original plan's Phase 6.

## 13.3 Schedule and owners (relative weeks)

| When | Work | Owner |
|---|---|---|
| Weeks 1–2 | Phases 0–3 (setup, data, baseline, engine) | done |
| Weeks 2–3 | Phase 4 pyramid accuracy (E5) | done |
| Week 3 | Phase 5: tile engine, scheduler, server (final plan Phases 1–2) | Swayam, done |
| Week 3 | Phase 5: web viewer and prefetch (final plan Phase 3) | Kartikeya, done |
| Week 3 | Phase 5: QGIS plugin (final plan Phase 4) | shared, done |
| Week 3 | Phase 5: E6 benchmark and tuning (final plan Phase 5) | Siddharth, done |
| End of Week 3 | Final report, slides, demo (final plan Phase 6) | all |

## 13.4 Open items

- The professor update (`docs/prof_update_phase2.md`) covers Phase 2 only and has not been sent; the Phase 3, 4 and 5 progress reports are ready in `report/`.
- Not done in Phase 5: the QGIS box-drawing baseline from the plan (QGIS's own rendering is covered by E1q); one look at the plugin in the QGIS GUI by hand.
- Possible improvements found but not done: a per-level correction factor for the slope underestimate; padding grids to a multiple of $2^k$ for exact overviews; caching decoded blocks across windows in the batch engine (the viewer's block cache already does this for tiles).

## 13.5 Questions to ask the professor

- Is "standard tools don't break, they become slow; after parallelising, data movement is the bottleneck" the kind of bottleneck answer he wants, or should we also benchmark other tools (WhiteboxTools, GRASS)?

- Does the debounced pan/zoom viewer with neighbour prefetch match what he asked for, and is the E6 method (a virtual client replaying seeded traces, cross-checked in headless Chrome) acceptable evidence for it?
- Is an honest "when the GPU helps and when it doesn't" (1.4× end to end, 50× kernel) the expected GPU result?
- Is the slope underestimate at coarse levels acceptable for display, if the viewer shows the expected error?

# 14 Presentation preparation

## 14.1 Numbers to remember

| Fact | Number |
|---|---|
| Largest grid | 10.8 billion cells, 43 GB float32, 2.6× RAM |
| gdaldem slope at 1 m | 208 s, 1 of 22 threads, 0.87 GB RAM, 73% computation |
| QGIS whole-area redraw at 1 m | 31.9 s without pyramid, 0.09 s with (360×) |
| Best storage layout | tiled ZSTD: 2.9× smaller, decodes 2.6× faster than LZW |
| Our CPU engine | 3.5× faster at 1 m (60 s), ∼1.4 GB flat, identical to gdaldem |
| Naive whole-grid NumPy | ∼24 B/cell, fails above ∼0.5–0.7 B cells |
| Our GPU engine | 43.8 s at 1 m (4.75× gdaldem); kernel 50× CPU; copies 95% of GPU time |
| GPU crossover | $512^2$ cells (slope), $256^2$ (three products) |
| Three products fused | 3.4–4.2× vs three gdaldem runs (CPU); 5.9× (GPU, 2 m real) |
| On-demand tile | 5–8 ms at every level; 8–450× faster than exact |
| Slope error per level | 2.3° (1 up), 4.4° (2), 5.6° (3), 6.4° (4), 10° (6); always too low |
| Aspect error | median 1.8–8.5° |
| Precomputed slope pyramid | 10.6 s, 1.0 GB per derivative |
| Runs measured | 1817 process runs (E1–E4) + E5 timings; 2 failures (intended out-of-memory); E6 1,068 viewer runs |
| *Phase 5: the viewer (E6)* | |
| Tile kernels in parallel | 5.4× on 8 threads (4.3× before moving quantising and PNG off the GIL) |
| Block cache | 64 tiles decode 100 blocks instead of up to 576; without it 6× more decoding, p95 +40% |
| E6 size | 24 configurations, 17 traces, 1 warm-up + 5 runs, fresh server per run: 1,068 runs |
| Panning with prefetch | 98–100% of new tiles already in the browser (without: 0% straight, ∼50% reversing); 0–5 foreground requests instead of 36–202 |
| Debounce (gate 2) | 62 → 19 server tiles per zoom gesture (3.3×); first exact tile 63–71 ms after the debounce |
| Why 150 ms | notched wheel: 100 ms gives 36 tiles (= no debounce), 150–250 ms give 15; 200 ms is 40–50 ms slower |
| Prefetch vs foreground (gate 1) | server p95 90 ms with vs 95 ms without (same tiles); isolation test +0.4%; only 3% of foreground tiles overlap a prefetch request |
| Prefetch cost | precision 0.20–0.56 (2–5 fetched per tile used); ∼3 E-core CPU-seconds per 5 s pan |

| Fact | Number |
|---|---|
| Latency after a zoom | first exact tile 57–89 ms after the commit, whole view 76–158 ms; box zoom 64 / 127 ms |
| Connection wait | client p95 ∼40 ms above server p95 (6 connections per browser) |
| Cancellation | aborted request dropped in a median 37 ms (polling gave 58–80 ms) |
| Server memory | never above 982 MiB (budget ≈2.2 GB); prefetch pauses below 1.0 GB free |
| Pinning held | 20,242 fg jobs on P-cores only, 122,215 bg jobs on E-cores only |
| Chrome vs virtual client | same commits and requests; view complete within 10% |
| QGIS plugin | full-resolution box identical to gdaldem (1.22 M cells, max difference 0); 13/13 checks |

### 14.2 Suggested slide order

1. Problem and goal (MoM): big DEM, one laptop, find the real bottleneck, show only what is needed.
2. Data: one area, real 90/30/8/2 m + synthetic 2/1 m; why synthetic is timing only.
3. Method: harness, cold/warm, pinning, stage split; why it is fair.
4. Baseline: gdaldem scaling, memory, stage split; QGIS with/without pyramid.
5. Our engine: pipeline diagram; speed-up and memory; naive breaking point; correctness.
6. GPU: kernel vs end to end; crossover; optimisation steps; "the bottleneck moves" chart.
7. Pyramid on demand: error by level and slope class, maps, time saved, trade-off chart.
8. The viewer: what the professor asked for (map-like pan/zoom, prefetch); architecture diagram (Figure 17); the design choices that matter (lattice on COG blocks, block cache, fg/bg on P/E cores, generations and cancellation, debounce, prefetch ring).
9. **Demo** (live, with the recorded video as fallback): pan and zoom in the browser (hillshade, then slope with the expected error shown), Shift + drag box zoom, exact mode; then one box in QGIS.
10. E6 method: virtual client, traces, protocol, the two gates; Chrome cross-check.
11. E6 results: pan hit rate vs speed; debounce chart and why 150 ms; gate 1 (prefetch does not slow the foreground); the 6-connection and block-decoding bottleneck.
12. The story and the answer to G1 and G2 (and the viewer as the practical result); limitations; what is left.

### 14.3 Likely questions and answers

| Question | Answer |
|---|---|
| Why not just give gdaldem more threads? | We tested `GDAL_NUM_THREADS=ALL_CPUS` and a 4 GB cache: 2–4% *slower*. Only decompression is parallel there; gdaldem's computation is single-threaded by design. |

| Question | Answer |
|---|---|
| Is the comparison fair? | Same machine, grids, harness and cores (12 P-threads); gdaldem at its fastest settings; engine times include Python start-up; results identical on every cell. |
| Why use upsampled 1 m data? | No free 1 m DEM exists for the area. Run time depends on cell count, which we verified on real vs upsampled data; accuracy uses real data only. |
| Why is the GPU only 1.4× faster? | The kernel is 50× faster, but 95% of the GPU's time is copying over PCIe and the run is limited by decompressing and writing. Amdahl's law. |
| When does the GPU make sense? | Windows $\geq 512^2$, several products at once, heavy operators, or results that stay in memory (display), not written to disk. |
| Why does coarse-level slope come out lower? | Averaging smooths the terrain before the gradient is taken; steep, rough slopes lose the most. |
| Is that error acceptable? | For display, yes (the map keeps every landform; error is shown). For thresholds (e.g. >45°), no: use the exact mode or a precomputed slope pyramid. |
| Why not precompute everything? | Exact and slightly faster, but a full pass and ∼1 GB per derivative and parameter; on demand needs no storage and supports any parameter. |
| Why did you drop box zoom as the main interaction? | The professor asked for map-like continuous pan and zoom. The fixed $512^2$ lattice keeps the cost per tile at the same few ms at every zoom (E5), so continuous viewing is possible. Box zoom remains as Shift + drag and as `POST /view` for QGIS. |
| Why debounce the zoom? | A wheel gesture sends many ticks; committing on each one starts and aborts a new view every 17 ms. The server then renders 62 foreground tiles per gesture instead of 19 (3.3× more), most of them for views the user never stops at. During the gesture the screen shows the tiles it has, rescaled, so nothing goes blank. |
| Why 150 ms and not 100 or 200? | With 60 Hz bursts 100–250 ms all give 19 tiles. A notched wheel (ticks 30–150 ms apart) separates them: 100 ms commits between notches (36 tiles, as bad as none); 150 ms and longer coalesce the gesture (15). 200 ms adds 40–50 ms of delay for no fewer tiles. |
| Why doesn't prefetch slow the foreground? | Three reasons: it runs on the efficiency cores while the foreground runs on the performance cores; a prefetch worker does not start a tile while any foreground tile is pending; and the client stops prefetch at the first input of a gesture and restarts it only when the view is complete (only 3% of foreground tiles ever overlap a prefetch request). On identical tiles server p95 is 90 ms with prefetch vs 95 ms without: it even helps, because it has already decoded shared blocks. Not yielding costs 9%. |
| Isn't prefetch wasteful? | Yes, 2–5 tiles fetched per tile used (precision 0.20–0.56), because a ring of 8 neighbours loads directions the user does not take. The cost is about 3 E-core CPU-seconds per 5 s pan; the gain is 98–100% of new tiles ready while panning instead of 0–50%. Velocity-first ordering keeps it useful at speed (0.99 vs 0.91). |

| Question | Answer |
|---|---|
| Why do the 6 connections matter? | A browser opens at most 6 HTTP/1.1 connections to one server; other tile requests wait. That wait is why client p95 is about 40 ms above server p95, and why 4, 6 or 8 server workers give the same client latency. Prefetch takes at most 2 connections, leaving 4 for visible tiles. We keep 8 workers so a second client (QGIS) does not queue. |
| Why not the GPU for the tiles? | A $512^2$ tile sits at the CPU/GPU crossover from E4 (GPU only 1.3× faster including the copies), and a tile's cost is mostly decoding compressed blocks, not the kernel. In E5 the 12-thread kernel took ∼0.5 ms of a 5–8 ms tile; the server instead runs one single-threaded kernel per tile (median 2.3 ms, p95 6.2 ms for foreground tiles in E6) on 8 tiles at once, which gives more throughput than threading inside one tile. E6 is CPU only; a GPU tile path is a stretch item. |
| How does the QGIS plugin work? | A panel and a box tool. The box is reprojected to UTM 44N and sent with the map's width to the same server's `POST /view` in a background `QgsTask`; the server picks the level that matches the box on screen (or full resolution) and returns a GeoTIFF that is added as a styled layer with level, cell size and expected E5 error in its metadata. A full-resolution box is identical to gdaldem's slope (max difference 0). |
| What happens with little free memory? | All caches are byte-budgeted (blocks 1024 MiB, tiles 512 MiB, browser 256 MiB; server peak in E6 982 MiB). Below 1.0 GB free RAM the server pauses prefetch and answers prefetch requests with 503 at once; the client backs off for 2 s. Visible tiles are always served. `POST /view` refuses boxes over 64 M cells. |
| How do you know the virtual client behaves like a browser? | It is a port of the viewer's code reading the same `policy.json`, with 6 connections and real aborts. We replayed three traces in real headless Chrome over the DevTools protocol: same commits, same foreground requests (15, 0, 35), same prefetched tiles, view complete within 10%, first frames 18–25 ms sooner in Chrome. So E6 latencies are slight upper bounds. |
| What happens when the user zooms away while tiles are computing? | The zoom commit raises the generation; the server drops older queued work of that session, the browser aborts requests no longer visible (the server sees the disconnect and cancels in a median 37 ms), and running tiles stop at their next stage check. |
| How do you know your results are correct? | 119 tests; every cell of four full grids compared with gdaldem; differences explained by gdaldem's float32 rounding and reproduced exactly. For the viewer: 161 tests in total; every auto tile equals the engine's result cell for cell; a QGIS full-resolution box equals gdaldem's slope exactly. |
| How repeatable are the timings? | Median IQR 1.2–1.6% of the median; cold cache verified; background interference detected and removed; disturbed attempts kept as evidence. |
| What would change on another laptop? | Absolute times; the shape of the results (where the bottleneck is) should hold on similar hardware; with a slow HDD, reading would matter more. |

## 15 How to reproduce everything, and where things are

```
conda activate geo
python datasets/prepare.py --synthetic            # rebuild all grids
powershell -ExecutionPolicy Bypass -File bench\run_phase2.ps1  # E1, E2, QGIS
powershell -ExecutionPolicy Bypass -File bench\run_e3.ps1      # E3 CPU
powershell -ExecutionPolicy Bypass -File bench\run_e4.ps1      # E4 GPU
python -m bench.experiments.e4_crossover
python -m bench.experiments.e5_timing --precompute
python -m bench.experiments.e5_accuracy
powershell -ExecutionPolicy Bypass -File bench\run_e5.ps1      # E5 timing
python -m pytest tests/ -q                                     # correctness
python -m tests.check_full_grid
python -m bench.analysis.phase2_report   # also: phase3_report,
                                         # e4_report, e5_report, run_stats
# Phase 5 (viewer, E6)
python -m server                         # http://127.0.0.1:8765/viewer/
python -m bench.check_tile_parallel      # parallel gate (Phase 1)
python -m bench.check_server             # isolation + cancellation gates
python -m bench.check_viewer             # in-browser self-test (Chrome)
powershell -ExecutionPolicy Bypass -File bench\run_e6.ps1      # E6, ~2.5 h
python -m bench.experiments.e6_view --only base debounce-off debounce-100
  debounce-150 debounce-250 maxwait-500 --traces zoom-notched
  zoom-notched-slow                      # notched traces (one line)
python -m bench.e6_browser               # headless Chrome vs virtual client
python -m bench.analysis.e6_report       # e6_tables.md, plots/e6_*.png
cd viz/web && node --test tests/viewer.test.js   # viewer unit tests
```

| Where | What |
|---|---|
| `docs/PROJECT_PLAN.md` | original plan, methodology (§6), phase status (its Phase 5–7 part is replaced by the final plan) |
| `docs/FINAL_PLAN.md` | working plan for the viewer and what is left (Phase 0 = done, Phases 1–5 viewer + E6, Phase 6 deliverables) |
| `docs/PHASE5_DEBOUNCED_ZOOM_SPEC.md` | the team's engineering spec for the viewer (goals, gates) |
| `docs/PHASE2_RESULTS.md`, `PHASE3_RESULTS.md`, `PHASE3_GPU_RESULTS.md`, `PHASE4_RESULTS.md`, `PHASE5_RESULTS.md` | detailed results per phase |
| `report/` | progress reports (Phase 2, 3, 4, 5) and this report |
| `results/runs.csv`, `results/e5_*.csv`, `results/e6_*.csv` | every measurement |
| `results/plots/`, `results/*_tables.md`, `results/run_stats.md` | charts, tables, spreads |
| `demlab/`, `bench/`, `tests/` | engine, tile engine and scheduler, harness and experiments, tests |
| `server/`, `viz/web/`, `qgis_plugin/` | tile server, web viewer (with `policy.json`), QGIS plugin |
| `old files/` (local only) | earlier scripts, superseded results, logs, with a README |
| GitHub | `https://github.com/Swayamgoyal/Geoguessers` (no data, no `old files`) |
| `K:\spatial project\ GeoGuessers_handoff` | complete verified copy: project, all data, environment, notes; start with `README_HANDOFF.md` |